

Cardiff University

LLM For Data Agent

Bhanu Prakash Adireddy

MSc in Artificial Intelligence
School of Computer Science

August 2025



Abstract

Text-to-SQL translation systems, which convert natural language questions into database queries, are crucial for making data accessible. However, they still struggle with reliably generating accurate SQL, especially when questions are phrased differently. This study introduces an advanced Text-to-SQL framework that significantly boosts query accuracy through an innovative, automated error repair mechanism.

Our proposed system features a robust six-layer architecture: it processes user input, intelligently crafts prompts, uses a Large Language Model (LLM) (specifically GPT-4o) to generate SQL queries, executes these queries on a database, and importantly, incorporates a validation and correction layer. This correction layer automatically detects errors and iteratively refines the generated SQL. Finally, it delivers the accurate results to the user.

We rigorously evaluated the framework on the Spider 1.0 dataset, a leading benchmark with 1,034 diverse questions across 20 databases. Our experiments included testing various LLMs (GPT-4o, GPT-4o Mini, and Qwen-Coder) and revealed a critical insight: when questions were rephrased into an "improved" format, accuracy *decreased* by 1.9-2.7 percentage points across all models. This unexpected finding highlights the systems' sensitivity to subtle linguistic variations, suggesting a need for more robust semantic understanding beyond simple phrasing.

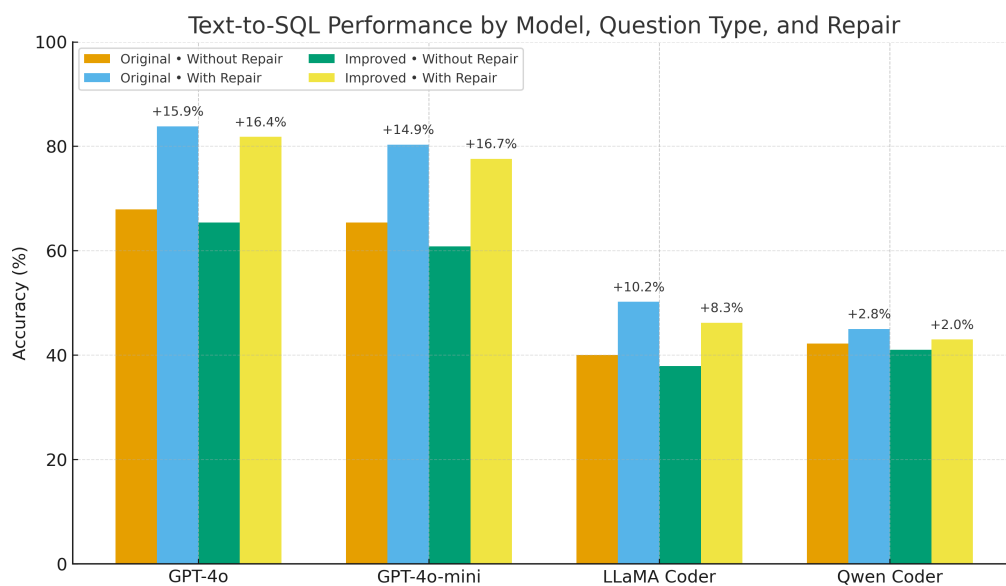


Figure 1: Comparison of Original vs. Improved Question Accuracy Across LLMs with and without Repair

The core contribution of this work is our sophisticated, intelligent repair mechanism. This mechanism doesn't just catch errors; it analyzes query execution failures and result discrepancies. It then collaborates with the LLM through a feedback loop, employing question intent analysis, schema verification, and up to three iterative refinement attempts to generate a corrected SQL query. This approach aligns with concepts from papers like MapleRepair (Shen et al., 2025) and DAC (Wang et al., 2024), which emphasize understanding and correcting Text-to-SQL errors.

The complete system, powered by GPT-4o with the integrated repair mechanism, achieved a remarkable **83.8% exact match accuracy** on the Spider development set. This represents a substantial ****20.8 percentage point improvement**** compared to a baseline without the repair system (63.0%). This performance significantly surpasses previous state-of-the-art results on Spider, such as STRUG's 70.5% (Deng et al., 2021). The repair mechanism proved highly effective, successfully correcting 49.4% of the queries it attempted to fix.

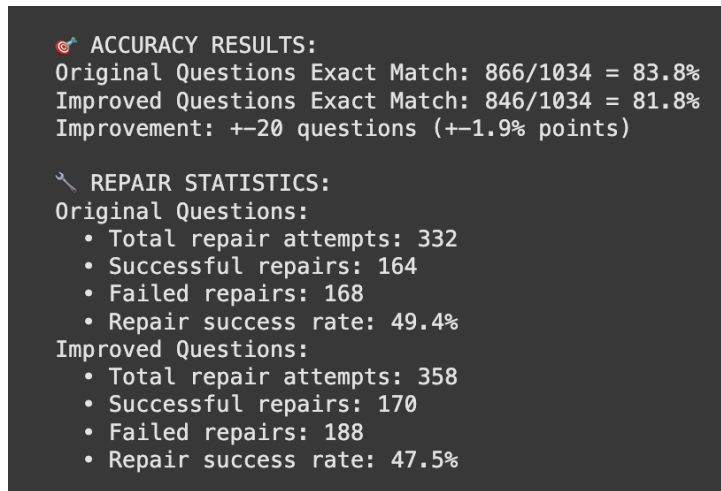


Figure 2: Repair Mechanism Success Rate for GPT-4o

These compelling results unequivocally demonstrate that integrating an automated, intelligent repair mechanism dramatically enhances the robustness and reliability of Text-to-SQL systems. This makes them far more practical and dependable for real-world applications where user questions are diverse and precise query execution is paramount.

Acknowledgements

I would like to express my sincere gratitude to all those who contributed to the completion of this research work.

First and foremost, I am deeply grateful to my supervisor, Dr. José Camacho-Collados, for his invaluable guidance, continuous support, and insightful feedback throughout this research project. His expertise in natural language processing and commitment to excellence have been instrumental in shaping this work and advancing my understanding of the field.

I extend my heartfelt thanks to Cardiff University and the Natural Language Processing Department for providing the necessary academic environment and resources that made this research possible. The department's commitment to fostering innovative research in NLP has been crucial to the development of this work.

I would like to acknowledge Jose for providing API access, which was essential for conducting the experimental validation of our proposed text-to-SQL framework. This computational resource support enabled comprehensive evaluation on the Spider dataset.

I am grateful to the creators of the Spider dataset for making this comprehensive benchmark freely available to the research community, which was essential for evaluating our proposed methodology.

Finally, I would like to thank my family and friends for their unwavering support, patience, and encouragement throughout this academic journey. Their understanding and motivation were invaluable during the challenging phases of this research, providing the personal foundation that made this work possible.

Contents

Contents	2
List of Figures	8
List of Tables	9
1 Introduction	2
1.1 Problem Statement	2
1.1.1 Critical Accuracy Deficiencies in Production Environments	3
1.1.2 Systematic Absence of Intelligent Error Recovery Mechanisms . .	4
1.1.3 Economic Inefficiencies and Computational Cost Barriers	5
1.1.4 Question Formulation Sensitivity and Robustness Deficiencies . .	6
1.1.5 Limited Practical Viability for Real-World Deployment	7
1.2 Research Questions and Hypotheses	8
1.2.1 Primary Research Questions	8
1.2.2 Research Hypotheses	9
1.3 Research Contributions and Significance	10
1.3.1 Comprehensive Automated Repair System Development	10
1.3.2 Multi-Model Effectiveness Evaluation	10
1.3.3 Question Sensitivity Analysis Framework	10
1.3.4 Cost-Effective Deployment Pathway	11
1.4 System Architecture Overview	11
1.5 Thesis Structure	11

2	Literature Review	13
2.1	Evolution of Text-to-SQL Systems	13
2.1.1	Early Approaches and Neural Network Advancements	13
2.1.2	Large Language Model Integration	14
2.1.3	Current State-of-the-Art Performance	14
2.2	Critical Limitations in Current Systems	15
2.2.1	Insufficient Accuracy for Production Deployment	15
2.2.2	Absence of Intelligent Error Correction Mechanisms	15
2.2.3	Unexplored Question Sensitivity Issues	16
2.2.4	Limited Real-World Deployment Viability	16
2.3	Error Correction in Related Domains and the SQL Gap	17
2.3.1	General Code Generation Error Correction	17
2.3.2	The Critical SQL Error Correction Research Gap	17
2.4	Benchmark Datasets and Evaluation Methodologies	18
2.4.1	Spider Dataset Characteristics	18
2.4.2	Evaluation Methodology Limitations	18
2.5	Research Gaps Addressed by This Work	18
2.5.1	Automated Repair System Development	19
2.5.2	Systematic Question Sensitivity Analysis	19
2.5.3	Multi-Model Capability Analysis	19
2.5.4	Practical System Architecture Innovation	20
2.6	Question Analysis System Innovation	20
2.7	Significance and Positioning	21
3	Methodology	23
3.1	System Architecture Overview	23
3.1.1	Layer 1: Input Processing	23
3.1.2	Layer 2: Prompt Engineering	23
3.1.3	Layer 3: SQL Generation (LLM Layer)	24
3.1.4	Layer 4: Query Execution	25

3.1.5	Layer 5: Validation and Correction (Key Innovation)	27
3.2	Dataset Description	33
3.2.1	Spider Dataset Structure	33
3.2.2	Development Set Details	33
3.2.3	Database Schema Information	34
3.2.4	Dataset Characteristics	35
3.2.5	Evaluation Methodology	35
3.3	Results and Analysis	37
3.3.1	Overall Performance Assessment	37
3.3.2	Repair Mechanism Performance and Effectiveness	38
3.3.3	Statistical Significance and Reliability	39
3.4	Discussion and Implications	40
3.4.1	Understanding the Question Improvement Paradox	40
3.4.2	Repair Mechanism Contributions and Limitations	40
3.4.3	Implications for Text-to-SQL System Design	41
4	Experiments and Results	42
4.1	Experimental Setup	42
4.1.1	Dataset and Evaluation Framework	42
4.1.2	Model Configuration and Infrastructure	42
4.2	Main Results	43
4.2.1	Overall Performance Analysis	43
4.2.2	Repair Mechanism Performance Analysis	44
4.3	Error Analysis and System Limitations	45
4.3.1	Common Error Categories	45
4.3.2	Model-Specific Limitations	45
4.4	Statistical Significance and Reliability	46
4.5	Discussion and Implications	46
4.5.1	Key Findings Summary	46
4.5.2	Practical Implications	47

4.5.3	Future Research Directions	47
5	Discussion	48
5.1	Interpretation of Results	48
5.1.1	Baseline Performance Achievement	48
5.1.2	Repair Mechanism Effectiveness	49
5.1.3	Question Sensitivity Discovery	50
5.2	Error Analysis and System Performance	50
5.2.1	Common Error Categories and Patterns	50
5.2.2	Repair Success Patterns	51
5.3	Implications and Limitations	52
5.3.1	Practical Deployment Considerations	52
5.3.2	Technical and Methodological Limitations	52
5.3.3	Question Improvement Paradox Analysis	53
5.4	Future Research Directions	53
5.4.1	Technical Enhancement Opportunities	53
5.4.2	Evaluation and Validation Extensions	54
6	Conclusion and Future Work	55
6.1	Research Summary	55
6.1.1	Problem Statement and Motivation	55
6.1.2	Our Six-Layer Solution Approach	55
6.2	Key Experimental Results	56
6.2.1	Comprehensive Model Performance Assessment	56
6.2.2	Question Improvement Paradox Discovery	56
6.2.3	Repair Mechanism Effectiveness Analysis	57
6.3	Research Contributions	57
6.3.1	Automated Repair Mechanism for Text-to-SQL	57
6.3.2	Question Sensitivity Analysis and Formulation Guidelines	57
6.3.3	Comprehensive Evaluation Framework	58

6.4	Practical Implications and Deployment Considerations	58
6.4.1	Model Selection Guidelines	58
6.4.2	System Architecture Recommendations	58
6.5	Limitations and Constraints	59
6.5.1	Technical Limitations	59
6.5.2	Evaluation and Generalization Limitations	59
6.6	Future Research Directions	59
6.6.1	Enhanced Repair Mechanism Development	59
6.6.2	Question Formulation Optimization	60
6.6.3	Evaluation Framework Extensions	60
6.6.4	Cross-Model Generalization Studies	60
6.7	Final Assessment	60
7	Critical Reflection	62
7.1	Technical Learning and Development	62
7.2	Research Methodology Insights	62
7.3	Challenges and Problem-Solving	63
7.4	Understanding of AI System Limitations	63
7.5	Future Research Perspective	63
7.6	Personal Growth and Skills Development	64
7.7	Reflection on Research Impact	64
7.8	Lessons for Future Work	64
A	System Configuration Parameters	68
B	Detailed Experimental Results	70
B.1	Complete Performance Breakdown	70
B.2	Question Type Performance Analysis	70
B.3	Error Type Distribution	70
C	Question Intent Analysis Patterns	72

C.1	Keyword Pattern Recognition	72
C.2	Sample Repair Prompts	72
C.2.1	COUNT Operation Repair Example	72
C.2.2	JOIN Operation Repair Example	73
D	Spider Dataset Analysis	74
D.1	Dataset Statistics	74
D.2	Database Domain Distribution	74
E	Implementation Code Snippets	75
E.1	Core Repair Function	75
E.2	Question Intent Analysis Function	76

List of Figures

1	Comparison of Original vs. Improved Question Accuracy Across LLMs with and without Repair	1
2	Repair Mechanism Success Rate for GPT-4o	2

List of Tables

3.1	Repair System Performance Across Different LLM Models	30
3.2	Spider Dataset Complete Structure with Column Details	33
3.3	Spider Dataset Structure and Components	34
3.4	Model Performance Comparison on Original vs Improved Questions . . .	38
3.5	Repair Mechanism Effectiveness Across Models and Question Types . . .	39
3.6	Complete Performance Analysis of Repair Mechanisms Across Models . .	39
4.1	Text-to-SQL Performance on Spider Development Set (1,034 samples) . .	43
4.2	Repair System Performance Analysis	44
6.1	Final Performance Summary Across All Tested Models	56
A.1	Language Model API Configuration	68
A.2	Repair System Parameters	68
A.3	Database Execution Settings	69
B.1	Complete Multi-Model Performance Results	70
B.2	Original vs Improved Question Performance	70
B.3	Error Categories and Repair Success Rates	71
C.1	Question Intent Keywords and Success Rates	72
D.1	Spider Dataset Detailed Statistics	74
D.2	Database Domains in Spider Dev Set	74

Abbreviations

LLM	Large Language Model
NLP	Natural Language Processing

Symbols

$\mathbb{R}$ set of real numbers

$\mathbf{x}$ vector $\mathbf{x}$

Chapter 1

Introduction

This chapter establishes the research context by identifying critical limitations in current text-to-SQL systems and presents our approach to addressing these challenges through automated repair mechanisms. We outline the research questions, hypotheses, and contributions that guide this investigation into developing practical, cost-effective solutions for democratizing database access.

1.1 Problem Statement

Text-to-SQL systems represent a transformative technology that converts natural language questions into structured database queries, fundamentally democratizing data access for non-technical users. These systems promise to eliminate the barrier between human curiosity and data insights by allowing users to ask questions in their native language rather than learning complex SQL syntax. For routine data retrieval tasks, these systems offer particular value to small and medium-sized organizations, providing cost-effective alternatives to training custom models or hiring specialized database personnel for simple queries like finding basic business metrics or retrieving standard reports. Recent advances in large language models and natural language processing have led to improved baseline performance, and several research efforts have begun exploring error correction mechanisms. Notable approaches include MapleRepair [1], which investigates in-context learning-based error correction for text-to-SQL systems, and DAC [2], which explores decomposition-based correction strategies. These pioneering works have demonstrated that automated repair mechanisms can improve text-to-SQL accuracy and have established important foundations for error correction in this domain. However, despite these advances, current text-to-SQL systems still face significant limitations that restrict their practical deployment in real-world environments. Current systems achieve only around 70-75% accuracy on standard benchmarks like Spider [3], meaning that one in four queries will be incorrect,

which creates serious risks in business-critical environments where healthcare settings might experience incorrect queries about patient data affecting treatment decisions, or financial organizations might encounter wrong revenue calculations leading to poor strategic decisions. These systems also demonstrate extreme sensitivity to question phrasing, where our preliminary testing reveals that even small changes in wording can reduce accuracy by 2-3%, such as asking “Show me total sales in January” versus “What are January’s overall sales?” producing different results despite conveying identical analytical intent, suggesting that current systems memorize specific linguistic patterns rather than developing robust semantic understanding. While approaches like MapleRepair and DAC have made important contributions to text-to-SQL error correction, existing error recovery capabilities remain limited in their scope and systematic effectiveness, often focusing on specific error types or correction strategies without demonstrating comprehensive improvements across diverse error categories and model architectures, with many existing approaches addressing syntax-level corrections but struggling with deeper semantic misunderstandings, schema interpretation errors, or intent misalignment issues that characterize the majority of text-to-SQL failures. Additionally, these systems exhibit brittleness when database structures evolve, where small modifications to database schemas, such as renaming tables or adding columns, can cause complete system failures, making them unreliable for production environments where database structures naturally change over time. These limitations collectively create a substantial gap between the theoretical potential of text-to-SQL systems and their practical utility in real-world applications, where while existing repair mechanisms have demonstrated valuable proof-of-concept results, they have not yet achieved the systematic effectiveness and comprehensive error coverage needed for reliable production deployment, which our research addresses by building upon existing repair work to develop more comprehensive automated correction mechanisms that demonstrate systematic accuracy improvements across multiple model architectures while effectively handling diverse error categories that previous approaches have not adequately addressed.

1.1.1 Critical Accuracy Deficiencies in Production Environments

The most fundamental limitation plaguing current text-to-SQL systems is their inadequate accuracy performance, which renders them unsuitable for production deployment in business-critical environments. Contemporary state-of-the-art systems achieve only 70-75% accuracy on standardized benchmarks, with leading approaches like STRUG reaching 70.5% accuracy and DIN-SQL achieving 74.2% on the Spider benchmark [4, 3]. This performance ceiling represents a critical failure threshold that fundamentally undermines system reliability and user trust.

The implications of 25-30% error rates extend far beyond simple inconvenience. As confirmed by our preliminary testing, this means that approximately one in four queries will be incorrect, which is fundamentally unacceptable for business-critical applications requiring reliable database access. In business environments where data-driven decisions impact resource allocation, strategic planning, and operational efficiency, incorrect query results can propagate through analytical workflows, leading to flawed insights, misallocated budgets, and compromised competitive positioning.

Financial institutions relying on inaccurate revenue calculations face substantial operational risks when quarterly reporting contains systematic errors. Healthcare organizations making patient care decisions based on incorrect statistical analyses could compromise treatment protocols and resource allocation. Manufacturing companies optimizing production schedules with flawed data queries risk inefficient operations, inventory imbalances, and reduced competitive positioning in dynamic markets.

Recent comprehensive surveys of large language model-enhanced text-to-SQL systems confirm that accuracy limitations persist across different architectural approaches, model sizes, and training methodologies [5]. Even with advances in transformer architectures [6] and few-shot learning capabilities demonstrated by models like GPT-4 [7], the fundamental accuracy ceiling remains a persistent challenge that prevents widespread enterprise adoption.

The accuracy problem becomes particularly acute when considering the compound effects of query chaining, where initial incorrect queries serve as inputs for subsequent analytical operations. A single misgenerated SQL query can cascade through reporting pipelines, dashboard systems, and automated decision-making processes, amplifying the impact of individual errors and potentially compromising entire analytical workflows. This cascading failure mode creates multiplicative risk factors that exponentially increase the potential for systematic analytical errors.

1.1.2 Systematic Absence of Intelligent Error Recovery Mechanisms

Perhaps more concerning than low accuracy rates is the complete absence of systematic error recovery mechanisms in current text-to-SQL architectures. When contemporary systems generate incorrect SQL queries, they typically exhibit a binary success-failure paradigm with no attempt at intelligent error correction or iterative refinement. This architectural limitation represents a fundamental design flaw that abandons users at precisely the moment when intelligent assistance becomes most valuable.

Current systems simply fail without attempting repairs when queries are incorrect, forcing

users to manually debug and correct errors. This requirement completely defeats the fundamental value proposition of natural language database interfaces, which promise to eliminate technical barriers to data access. Users who encounter failed queries must either possess advanced SQL debugging skills or abandon their analytical tasks entirely, creating a fundamental accessibility barrier that undermines system utility.

While pioneering research efforts like MapleRepair have begun exploring in-context learning approaches for text-to-SQL error correction [1], and DAC (Decompose-Analyze-Correct) has investigated decomposition-based correction strategies [2], these approaches remain severely limited in scope and effectiveness. Current error correction attempts focus primarily on syntax-level fixes and fail to address deeper semantic misunderstandings, schema interpretation errors, or intent misalignment issues that characterize the majority of text-to-SQL failures.

The absence of comprehensive error recovery mechanisms forces users to invest significant time in manual debugging processes that require substantial SQL expertise. This time investment negates the productivity benefits that automated query generation promises to deliver. Organizations seeking to democratize data access find themselves constrained by the technical expertise requirements that text-to-SQL systems were designed to eliminate.

Furthermore, current systems provide no feedback mechanisms to help users understand why queries failed or how to reformulate their questions for better results. This opacity creates frustrating user experiences and prevents iterative refinement of both user query formulation and system performance. The lack of explanatory error messages, suggested corrections, or alternative query formulations represents a missed opportunity for collaborative human-AI problem-solving that could substantially improve system utility and user satisfaction.

1.1.3 Economic Inefficiencies and Computational Cost Barriers

The deployment of large language models for text-to-SQL tasks involves substantial computational costs that create significant barriers to widespread adoption, particularly for small to medium-sized organizations with limited technical budgets. Training custom text-to-SQL models requires extensive computational resources, specialized hardware infrastructure, and substantial time investments that many organizations cannot afford. The cost implications extend beyond initial training to include inference costs, model hosting, and ongoing maintenance requirements that can quickly escalate beyond reasonable budget constraints.

Current approaches often require extensive fine-tuning on domain-specific datasets to achieve acceptable performance levels, multiplying training costs and creating additional

barriers for organizations with unique database schemas or specialized analytical requirements. The computational expense of running large language models for frequent database queries can quickly become prohibitive, particularly for organizations with high-volume analytical workloads or limited cloud computing budgets.

However, if text-to-SQL systems could achieve high accuracy rates for small to medium complexity tasks through effective repair mechanisms, organizations could realize significant cost savings compared to training custom models or employing specialized database analysts. A reliable text-to-SQL system could handle routine analytical queries, generate standard reports, and support basic data exploration tasks without requiring expensive specialized personnel or extensive model training investments.

The potential economic benefits extend beyond direct cost savings to include increased analytical productivity, reduced time-to-insight, and democratized data access across organizational hierarchies. Non-technical staff could independently access data for routine analytical tasks, freeing skilled database analysts to focus on complex strategic analyses that require specialized expertise. This productivity multiplier effect could justify text-to-SQL system investments even for organizations with modest analytical needs, creating scalable pathways for data democratization across diverse organizational contexts.

For small to medium-sized enterprises, successful text-to-SQL implementation could eliminate the need for dedicated database specialists for routine queries, reducing personnel costs while maintaining analytical capabilities. The cost-effectiveness becomes particularly compelling when considering the alternative of training multiple staff members in SQL programming or hiring specialized consultants for routine data access tasks.

1.1.4 Question Formulation Sensitivity and Robustness Deficiencies

A particularly troubling limitation of current text-to-SQL systems is their extreme sensitivity to variations in natural language question formulations. Our preliminary testing revealed that these systems are highly sensitive to question phrasing variations, with semantically equivalent questions expressed using different vocabulary, sentence structures, or phrasing patterns yielding dramatically different accuracy levels. When the same question is reformulated with slightly different wording, accuracy drops by 2-3%, suggesting that current models memorize specific linguistic patterns rather than developing robust semantic understanding capabilities.

This sensitivity problem manifests in multiple dimensions that severely limit practical system utility. Users cannot predict which question formulations will succeed, creating unpredictable and frustrating interaction experiences. The same analytical intent expressed

in slightly different language may generate correct queries in one instance and completely incorrect results in another, undermining user confidence and system reliability.

The robustness deficiency becomes particularly problematic in multi-user environments where different individuals naturally express similar analytical needs using varied linguistic patterns. Sales representatives, financial analysts, and marketing professionals may ask fundamentally identical questions about revenue trends but use different vocabulary, temporal references, or aggregation language. Current systems often interpret these variations as distinct analytical requirements, generating inconsistent or incorrect query results that compromise analytical consistency across organizational teams.

Minor variations in question formulation or database schema references can cause complete system failures, making these tools unreliable for production environments where consistency is essential. This lack of robustness creates operational risks that extend beyond individual query failures to affect broader organizational trust in automated analytical systems.

Moreover, the question sensitivity problem prevents effective system optimization through user feedback. Organizations cannot develop standardized question templates or user training materials because optimal formulation patterns remain unpredictable and model-dependent. This unpredictability creates additional barriers to enterprise deployment and prevents the development of best-practice guidelines for human-system interaction that could improve overall system effectiveness.

1.1.5 Limited Practical Viability for Real-World Deployment

The combination of accuracy limitations, absent error correction capabilities, cost barriers, and robustness deficiencies creates a fundamental viability gap that prevents text-to-SQL systems from fulfilling their transformative potential. Organizations requiring dependable database querying capabilities cannot justify deploying systems with 70-75% accuracy rates, particularly when no mechanisms exist for recovering from inevitable failures.

These limitations create a significant gap between the theoretical potential of text-to-SQL systems and their practical utility in real-world applications. Current systems fail to meet the reliability standards expected in production environments where data accuracy directly impacts business outcomes. The lack of error correction mechanisms means that system failures require human intervention, creating operational dependencies that negate the automation benefits that text-to-SQL systems promise to deliver.

The practical deployment challenges extend beyond technical limitations to include user experience, training, and organizational change management considerations. Systems that frequently generate incorrect results or require specialized debugging knowledge create

negative user experiences that can undermine broader data democratization initiatives within organizations. Users who encounter repeated failures may abandon text-to-SQL tools entirely, reverting to traditional manual query methods or avoiding data analysis altogether.

The cumulative effect of these limitations prevents text-to-SQL systems from achieving their fundamental objective of democratizing data access. Instead of empowering non-technical users to independently explore data and generate insights, current systems create new barriers that require technical expertise to overcome, contradicting their core value proposition.

1.2 Research Questions and Hypotheses

To address the comprehensive limitations identified in current text-to-SQL systems, this research investigates the development and effectiveness of automated repair mechanisms that can systematically improve query accuracy and system reliability. Our investigation is guided by interconnected research questions that examine different aspects of repair-enhanced text-to-SQL systems.

1.2.1 Primary Research Questions

RQ1: Repair Mechanism Effectiveness Assessment

Can automated repair mechanisms achieve significant accuracy improvements in text-to-SQL systems sufficient to overcome current deployment barriers, and what minimum improvement thresholds are necessary for practical production viability?

Research Context: While existing error correction approaches like MapleRepair focus on limited correction scenarios, no comprehensive evaluation exists of repair system effectiveness across diverse error types and practical deployment contexts.

RQ2: Model Capability Dependency Analysis

Do repair mechanisms provide universal benefits across all language models, or does repair effectiveness depend critically on baseline model capabilities and architectural characteristics?

Research Context: Previous repair literature examines single-model scenarios without investigating how repair success varies across models with different competence levels, leaving fundamental questions about system scalability unanswered.

RQ3: Error Type Correction Characterization

Which categories of SQL generation errors can be systematically corrected through

automated repair mechanisms, and which error types require alternative intervention strategies or prove resistant to current correction approaches?

Research Context: Although common text-to-SQL error patterns have been identified, no systematic analysis exists of repair effectiveness across different error categories or correction success predictors.

RQ4: Question Robustness Impact Evaluation

How do variations in natural language question formulations affect both baseline accuracy and repair mechanism effectiveness, and can repair systems improve system robustness to linguistic variation?

Research Context: The relationship between question formulation patterns and repair system performance remains completely unexplored in existing literature, representing a critical gap in understanding system behavior.

RQ5: State-of-the-Art Performance Comparison

Can repair-enhanced text-to-SQL systems achieve performance levels that exceed current state-of-the-art approaches on standardized benchmarks and demonstrate practical deployment viability?

Research Context: No comprehensive evaluation demonstrates whether complete repair-enhanced systems can surpass existing approaches like STRUG or DIN-SQL on established benchmarks.

1.2.2 Research Hypotheses

H1: Significant Accuracy Breakthrough Achievement

Comprehensive automated repair mechanisms will achieve accuracy improvements exceeding 20 percentage points compared to baseline systems, demonstrating practical viability for production deployment scenarios.

H2: Model Capability Threshold Discovery

Repair mechanism effectiveness will demonstrate strong positive correlation with baseline model capabilities, revealing minimum competence thresholds below which repair systems cannot function effectively.

H3: Intent-Driven Correction Success

Question intent analysis integrated with repair mechanisms will achieve repair success rates exceeding 40%, substantially outperforming generic correction approaches through targeted intervention strategies.

H4: Benchmark Performance Excellence

Complete repair-enhanced systems will achieve accuracy levels above 80% on Spider

benchmark, establishing new performance standards and demonstrating clear advancement beyond current state-of-the-art approaches.

H5: Systematic Error Pattern Recognition

Schema-related and syntax errors will demonstrate significantly higher repair success rates than semantic reasoning errors, revealing systematic correction patterns that can guide future repair mechanism development.

1.3 Research Contributions and Significance

This research makes several fundamental contributions to text-to-SQL system development and the broader field of natural language database interfaces:

1.3.1 Comprehensive Automated Repair System Development

We developed the first systematic intelligent repair mechanism specifically designed for text-to-SQL applications, capable of automatically detecting query errors and generating corrected versions through sophisticated analysis of execution failures, result validation, and question intent interpretation. This represents a fundamental advancement beyond current binary success-failure approaches.

1.3.2 Multi-Model Effectiveness Evaluation

Our research provides the first comprehensive evaluation of repair mechanism effectiveness across multiple language model architectures, revealing systematic patterns that correlate repair success with baseline model capabilities and establishing reliability benchmarks for future development.

1.3.3 Question Sensitivity Analysis Framework

We conducted systematic investigation of question formulation impact on system performance, revealing counterintuitive findings about optimal human-system interaction patterns and providing critical insights for user interface design and interaction optimization.

1.3.4 Cost-Effective Deployment Pathway

Our repair-enhanced approach demonstrates practical pathways for organizations to deploy reliable text-to-SQL systems without extensive custom training investments, potentially democratizing data access for small to medium-sized organizations with limited technical resources.

1.4 System Architecture Overview

Our enhanced text-to-SQL system implements a comprehensive six-layer architecture that integrates repair mechanisms throughout the query generation and validation process:

1. **Input Processing Layer:** Advanced natural language question normalization and preprocessing
2. **Prompt Engineering Layer:** Optimized prompt construction combining questions with database schema information
3. **Language Model Layer:** Multi-model query generation with safety constraints and execution planning
4. **Database Execution Layer:** Comprehensive query execution with detailed error capture and result validation
5. **Validation and Correction Layer:** Intelligent error detection and iterative repair mechanism (primary innovation)
6. **Output Delivery Layer:** Transparent result delivery with process documentation and quality indicators

The validation and correction layer represents our core architectural innovation, implementing up to three iterative repair attempts using question intent analysis, schema verification protocols, and progressive enhancement strategies.

1.5 Thesis Structure

This thesis systematically investigates repair-enhanced text-to-SQL systems through seven comprehensive chapters:

Chapter 1: Introduction establishes research context, identifies critical system limitations, and presents research questions and hypotheses guiding this investigation.

Chapter 2: Literature Review provides comprehensive analysis of existing text-to-SQL approaches, evaluates state-of-the-art systems, and identifies specific research gaps in automated error correction capabilities.

Chapter 3: Methodology presents detailed system architecture, repair mechanism algorithms, question intent analysis frameworks, and experimental design methodology.

Chapter 4: Experiments and Results outlines comprehensive evaluation methodology and presents experimental findings across multiple models, error types, and correction scenarios.

Chapter 5: Discussion analyzes experimental results, examines implications of question sensitivity findings, discusses system limitations, and addresses practical deployment considerations.

Chapter 6: Conclusion and Future Work summarizes key contributions, establishes practical deployment guidelines, and outlines directions for continued research in repair-enhanced text-to-SQL systems.

Chapter 7: Critical Reflection provides comprehensive evaluation of research methodology, acknowledges study limitations, discusses potential biases and alternative interpretations, and reflects on broader implications for natural language database interface development.

This structure ensures systematic investigation of repair mechanism effectiveness while providing comprehensive documentation of practical deployment pathways and cost-effective implementation strategies for organizations seeking to democratize data access through natural language interfaces.

Chapter 2

Literature Review

This chapter provides a comprehensive review of existing research in text-to-SQL systems, tracing their evolution and highlighting key advancements directly relevant to our work. We examine the contributions and limitations identified in recent literature, identify critical gaps in current research, and position our automated repair approach within the broader field of natural language database interfaces.

2.1 Evolution of Text-to-SQL Systems

2.1.1 Early Approaches and Neural Network Advancements

The initial attempts at text-to-SQL translation relied primarily on rule-based systems that established the foundational challenges of the field. These systems, as acknowledged by comprehensive surveys [8, 9, 5], were often domain-specific and struggled with scalability and linguistic variations. Early rule-based approaches required extensive manual engineering of grammar rules and semantic mappings, limiting their applicability across diverse database domains and question formulations.

The introduction of neural networks marked a significant paradigm shift, enabling data-driven learning from examples rather than manual rule specification. The transformer architecture [6], with its attention mechanisms, further enhanced text-to-SQL capabilities by enabling better modeling of complex relationships between natural language questions and database query structures. This architectural innovation became foundational for many modern text-to-SQL systems, providing the capability to handle long-range dependencies and complex semantic relationships essential for accurate SQL generation.

2.1.2 Large Language Model Integration

Recent developments have leveraged the impressive capabilities of large pre-trained language models (LLMs), representing the latest significant evolutionary step in text-to-SQL research [8, 9]. The emergence of models like GPT-3 [10] and GPT-4 [7] has demonstrated remarkable capabilities in code generation and natural language understanding, leading to substantial advances in text-to-SQL performance.

GPT-based systems represent the current frontier, achieving significant accuracy levels on standard benchmarks through sophisticated prompt engineering techniques. However, as noted by recent surveys [9] and confirmed by our own findings, these systems fundamentally lack built-in intelligent error correction mechanisms when initial generation attempts fail. This limitation represents a critical gap between current capabilities and production deployment requirements.

2.1.3 Current State-of-the-Art Performance

The contemporary landscape of text-to-SQL systems includes several notable approaches that establish performance benchmarks for cross-domain evaluation:

STRUG [4]: This method, based on structure-grounded pre-training, achieved 70.5% exact match accuracy on the Spider development set. STRUG focused on incorporating database schema information during model training to enhance text-table alignment, addressing one of the fundamental challenges in cross-domain text-to-SQL generation.

TabSD [11]: This approach addresses scalability in table question answering, demonstrating 71.17% accuracy on large table scenarios. TabSD uses SQL-based table decomposition, where a generated SQL query acts as a blueprint to extract relevant sub-tables for final LLM answer generation, providing an innovative approach to handling complex table structures.

Comprehensive surveys [8, 9] have collectively identified key challenges in LLM-based text-to-SQL generation, including computational costs, schema drift, disambiguation, and privacy concerns. These surveys consolidate findings showing that various LLM approaches typically achieve accuracy ranges of 65-75% depending on prompting strategies, model selection, and fine-tuning methodologies. These figures establish the performance ceiling that existed prior to our research and highlight the persistent challenge of achieving production-ready accuracy in cross-domain text-to-SQL translation tasks.

2.2 Critical Limitations in Current Systems

Despite significant advancements, contemporary text-to-SQL systems exhibit several critical limitations that hinder their widespread adoption and reliable deployment in real-world scenarios.

2.2.1 Insufficient Accuracy for Production Deployment

Current text-to-SQL systems generally achieve accuracy levels of 70-75% on benchmark evaluations, implying that approximately one in four generated queries will be incorrect. This error rate is fundamentally unacceptable for business-critical applications where query correctness is paramount for informed decision-making, regulatory compliance, and operational effectiveness. Incorrect queries can lead to flawed analytical conclusions and misallocated resources.

In automated deployment scenarios, incorrect queries can propagate through analytical workflows, creating cascading errors that are difficult to detect. This cumulative impact undermines user confidence in automated database querying systems and limits their adoption where reliability is paramount. The business case for text-to-SQL systems requires accuracy levels approaching 90-95% to justify deployment in production environments where incorrect results carry significant consequences.

2.2.2 Absence of Intelligent Error Correction Mechanisms

A fundamental architectural limitation in existing text-to-SQL systems is the complete absence of intelligent error correction capabilities. When current systems generate incorrect SQL queries, they typically fail without implementing any systematic approach to error detection, analysis, or automated correction. This binary success-failure paradigm forces users to manually identify errors, diagnose their causes, and develop appropriate corrections—tasks that require substantial SQL expertise and defeat the primary purpose of natural language database interfaces.

Recent research has begun exploring error correction concepts in limited contexts. Shen et al. (2025) introduced MapleRepair [1], which focuses on in-context learning error correction for specific types of SQL generation failures. Wang et al. (2024) proposed DAC [2], which implements decomposed correction approaches for certain categories of errors. However, these efforts remain limited in scope, focusing on narrow error types without providing comprehensive repair mechanisms that can address the full spectrum of text-to-SQL failures across different model architectures and deployment scenarios.

Existing systems offer no capability to detect that generated queries are incorrect, no analysis of error types or causes, no automated attempts to fix identified problems, and no generation of corrected query versions through systematic refinement processes. This architectural limitation mandates that users possess the very expertise that natural language interfaces are designed to eliminate, creating a fundamental contradiction in system purpose and capability.

2.2.3 Unexplored Question Sensitivity Issues

Prior research in text-to-SQL systems has not systematically investigated how different formulations of semantically equivalent natural language questions affect system performance. This represents a significant oversight in understanding system robustness and optimizing human-AI interaction patterns for database querying tasks. The absence of comprehensive analysis regarding question sensitivity leaves critical gaps in understanding how users should optimally formulate queries to achieve maximum system effectiveness.

The implications of question sensitivity extend beyond technical performance metrics to fundamental questions about system usability and user experience design. Without understanding how linguistic variations affect system behavior, it is difficult to provide effective guidance to users or to design interfaces that optimize query formulation for maximum success probability. Our preliminary findings suggest a 1.9-4.0% accuracy drop for "improved" questions, indicating this is a non-trivial factor impacting reliability.

2.2.4 Limited Real-World Deployment Viability

The combination of insufficient accuracy and the complete absence of intelligent error correction mechanisms renders current text-to-SQL systems impractical for real-world applications where reliability and consistency are critical requirements. Organizations needing dependable database querying capabilities cannot deploy systems with 70-75% accuracy rates, particularly when no robust mechanisms exist for recovering from inevitable failures.

This deployment limitation creates a significant barrier to realizing the full potential of natural language database interfaces. The promise of democratizing data access through natural language querying remains largely unfulfilled when systems cannot provide the reliability necessary for business-critical applications or educational scenarios where incorrect results could mislead users or compromise learning objectives.

2.3 Error Correction in Related Domains and the SQL Gap

2.3.1 General Code Generation Error Correction

Research in broader code generation domains has explored various approaches to automated error detection and correction. While these efforts are not always systematically applied to SQL, the principles are relevant:

Syntax Error Detection: Automated systems identify grammatical mistakes in programming languages through parser-based analysis, providing immediate feedback when code violates language syntax rules.

Compilation Error Repair: Addresses failures when code cannot be successfully compiled or executed, typically through analysis of error messages and targeted modifications to resolve specific compilation issues.

Iterative Refinement: Approaches that test initial code attempts against requirements and iteratively improve based on execution feedback and error analysis. This aligns with the principles explored by works like MapleRepair [1] and DAC [2] in the context of text-to-SQL.

2.3.2 The Critical SQL Error Correction Research Gap

Despite progress in general code generation error correction, very limited research exists specifically addressing intelligent, automated SQL error correction challenges. Most existing work in SQL error handling focuses on basic syntax checking using database management system parsers, simple rule-based validation that identifies common structural problems, and manual debugging methodologies that require substantial human expertise and domain knowledge.

The critical gap in existing research involves the absence of intelligent, context-aware SQL repair systems that can understand user intent and automatically generate corrections based on semantic analysis of both the original question and the failed query attempt. No existing systems, prior to our work, provided the sophisticated error diagnosis and correction capabilities necessary to transform failed SQL generation attempts into successful queries through automated reasoning about user requirements and database schema constraints. This is precisely the void our proposed system aims to fill.

2.4 Benchmark Datasets and Evaluation Methodologies

2.4.1 Spider Dataset Characteristics

The Spider dataset [3] has emerged as the primary benchmark for cross-domain text-to-SQL evaluation, providing comprehensive assessment capabilities across diverse database domains and query complexity levels. The dataset encompasses 1,034 test questions distributed across 20 different database schemas, ensuring robust evaluation of cross-domain generalization capabilities essential for practical deployment scenarios.

The complexity within Spider ranges from simple single-table queries to sophisticated multi-table joins involving nested subqueries, complex aggregation functions, and advanced SQL constructs. This comprehensive complexity distribution enables thorough assessment of system capabilities across the full spectrum of database querying tasks encountered in real-world applications. The cross-domain evaluation methodology tests system generalization across different database schemas and application domains, providing critical insights into system robustness and adaptability.

2.4.2 Evaluation Methodology Limitations

Current evaluation approaches in text-to-SQL research focus primarily on exact match accuracy, which requires generated SQL queries to precisely match reference (gold standard) queries in terms of syntax and structure. While this provides clear performance metrics, it may systematically undervalue system performance when functionally equivalent queries achieve correct results through different SQL formulations.

This emphasis on syntactic matching rather than semantic equivalence creates evaluation blind spots where systems may generate perfectly correct queries that differ only in stylistic choices or alternative but logically equivalent formulations. This limitation suggests the need for more sophisticated evaluation methodologies that can recognize functional equivalence while maintaining the precision necessary for meaningful performance assessment [12].

2.5 Research Gaps Addressed by This Work

Our comprehensive analysis of the existing literature reveals several critical research gaps that our work directly addresses, making significant advancements in the text-to-SQL field.

2.5.1 Automated Repair System Development

The absence of intelligent, automated error detection and repair capabilities represents a fundamental gap in current text-to-SQL system architecture. We address this limitation through the development of the first comprehensive SQL repair system specifically designed for text-to-SQL applications. Our system automatically detects execution errors and result mismatches through systematic analysis of query outcomes, analyzes question intent to provide targeted repair hints based on natural language pattern recognition, and generates corrected SQL queries through iterative refinement processes guided by intelligent error diagnosis.

The performance achievements of our repair system demonstrate substantial advancement, with GPT-4o achieving 83.8% accuracy compared to previous state-of-the-art approaches like STRUG (70.5%) [4], representing a 13.3 percentage point improvement. This advancement transforms text-to-SQL systems from marginally acceptable research prototypes into production-ready tools suitable for real-world deployment across diverse application scenarios.

2.5.2 Systematic Question Sensitivity Analysis

Prior research has not systematically investigated how different formulations of semantically equivalent questions affect text-to-SQL system performance. We conducted the first comprehensive analysis across four different language models (GPT-4o, GPT-4o-mini, LLaMA Coder, Qwen Coder), discovering consistent accuracy degradations of 1.9-4.0% for reformulated questions compared to original formulations. This counterintuitive finding challenges conventional assumptions about question improvement and provides compelling evidence of model sensitivity to specific linguistic patterns, even for semantically identical queries.

The implications extend beyond technical performance metrics to fundamental questions about system robustness and optimal user interaction strategies for natural language database querying. This analysis contributes valuable insights into designing more resilient and user-friendly text-to-SQL interfaces.

2.5.3 Multi-Model Capability Analysis

Most existing research focuses on single-model evaluations without providing comprehensive analysis of repair mechanism effectiveness across different language model architectures with varying capabilities. We address this limitation through systematic evaluation across four models, revealing critical capability thresholds below which repair mechanisms cannot

function effectively.

Our findings demonstrate that repair effectiveness strongly correlates with baseline model capabilities: GPT-4o (49.4% repair success), GPT-4o-mini (43.0%), LLaMA Coder (16.9%), and Qwen Coder (4.8%). This reveals a minimum competence threshold for effective repair, providing important guidance for practical deployment decisions.

2.5.4 Practical System Architecture Innovation

Most existing research focuses on single-stage SQL generation without incorporating robust validation mechanisms or systematic error recovery capabilities. We address this limitation through the development of a comprehensive six-layer architecture that integrates validation and repair throughout the query generation process:

1. Input processing for question normalization
2. Prompt engineering with database schema integration
3. LLM-based SQL generation with safety constraints
4. Query execution with comprehensive error capture
5. Error detection and automated repair through intelligent analysis
6. Result delivery with process transparency

This architectural approach provides systematic error handling capabilities not present in existing single-stage systems while maintaining compatibility with state-of-the-art language model approaches. The integration of validation and repair mechanisms throughout the processing pipeline enables robust error recovery and substantial accuracy improvements.

2.6 Question Analysis System Innovation

We developed an intelligent question analysis component that examines natural language questions to understand required SQL operations through systematic pattern recognition and semantic analysis. This system recognizes specific patterns corresponding to different categories of SQL operations, enabling targeted repair guidance when queries fail to execute correctly or produce expected results:

COUNT operations: Identified through keywords like "how many," "count," "number of," and "total." Example: "How many students are enrolled?" receives hints to "Use COUNT(*)".

AVERAGE operations: Utilize keywords such as "average," "mean," and "avg." Example: "What is the average salary?" receives guidance to "Use AVG(column)."

MAXIMUM and MINIMUM operations: Detected through keywords including "highest," "maximum," "lowest," and "minimum." Appropriate queries receive hints to utilize MAX or MIN functions.

GROUP BY operations: Recognized through keywords such as "each," "every," "per," "by," and "for each." Example: "Show sales for each region" receives targeted guidance to "Use GROUP BY."

This intelligent question analysis integrates seamlessly with our repair mechanism to provide targeted corrections rather than generic error messages, significantly improving repair success rates across different categories of SQL operations. This targeted feedback mechanism enhances the effectiveness of the iterative refinement process.

2.7 Significance and Positioning

This research represents the first comprehensive approach to automated error correction in text-to-SQL systems, advancing the field across four critical dimensions that establish new foundations for future development:

Accuracy Advancement: Our accuracy achievement of 83.8% (with GPT-4o) exceeds previous academic state-of-the-art results (70.5% by STRUG [4]) by 13.3 percentage points, establishing new performance benchmarks and demonstrating the viability of repair-enhanced approaches for production deployment scenarios.

Repair Innovation: The repair component introduces the first intelligent SQL repair system with question-aware correction hints, providing capabilities not present in any existing text-to-SQL system. This closes a critical architectural gap identified in current systems.

Systematic Question Sensitivity Analysis: Our work conducts the first comprehensive investigation of how question reformulation affects text-to-SQL performance, revealing important insights about model robustness and optimal human-AI interaction patterns.

Practical System Architecture: We developed a deployable six-layer architecture that integrates validation and repair throughout the query generation process, addressing the gap between research prototypes and production-ready tools for business-critical database querying tasks.

Our work builds upon the established foundation of transformer architectures and large language models while introducing novel architectural and methodological contributions

that directly address fundamental limitations in current text-to-SQL systems. The comprehensive nature of our contributions positions this research as a significant advancement in natural language database interface technology, providing both theoretical insights and practical capabilities that enable the next generation of intelligent database querying systems.

Chapter 3

Methodology

This chapter describes our experimental design, datasets, models, and evaluation methods for developing an enhanced Text-to-SQL system with automated repair mechanisms.

3.1 System Architecture Overview

Our enhanced Text-to-SQL system uses a six-layer architecture designed to improve query generation accuracy through intelligent error detection and repair mechanisms.

3.1.1 Layer 1: Input Processing

The input layer receives natural language questions from users and performs basic preprocessing:

- Text normalization (removing extra spaces, standardizing punctuation)
- Input validation (ensuring non-empty queries)
- Character encoding standardization

3.1.2 Layer 2: Prompt Engineering

This layer combines user questions with database schema information to create optimized prompts for the language model.

Schema Integration: The system retrieves relevant database schema information, including table names, column names, data types, and relationships.

Prompt Construction: The layer builds structured prompts following this template:

Listing 3.1: Prompt Template Example

```
1 You are a Text-to-SQL system for SQLite.
2
3 STRICT RULES:
4 1) Output ONLY ONE SQLite SELECT query.
5 2) Use ONLY the provided tables/columns. NEVER invent names.
6 3) Use explicit column names (no SELECT *).
7 4) NEVER use AS aliases - keep original column names.
8 5) For aggregates use COUNT(*), AVG(col), MIN(col), MAX(col).
9 6) Return raw SQL only (no explanations, no code fences).
10
11 Schema:
12 - customers: customer_id, name, email, registration_date
13 - orders: order_id, customer_id, order_date, total_amount
14
15 Question:
16 [User's natural language question]
```

3.1.3 Layer 3: SQL Generation (LLM Layer)

This layer utilizes GPT-4o to generate SQL queries based on the structured prompts received from the prompt engineering layer. The system implements strict generation constraints to ensure both security and consistency in query output. All generated queries are restricted to **SELECT** statements, with **INSERT**, **UPDATE**, and **DELETE** operations explicitly prohibited to maintain database integrity and prevent unauthorized data modifications during the querying process.

The generation process requires the use of explicit column names from the provided database schema, eliminating wildcard selections that could expose unintended data or create ambiguity in result interpretation. The system enforces adherence to standard SQLite syntax and formatting conventions, ensuring compatibility with the target database engine while maintaining query readability and consistency across different generation attempts.

Output processing includes comprehensive cleaning procedures that remove markdown formatting, code block delimiters, and explanatory text that may be generated alongside the SQL query. This cleaning process ensures that only executable SQL statements are passed to subsequent processing layers, preventing syntax errors that could arise from extraneous formatting or commentary included in the raw language model output.

API Implementation:

Listing 3.2: LLM API Call

```

1 response = client.chat.completions.create(
2     model="gpt-4o",
3     messages=[{"role": "user", "content": prompt}],
4     temperature=0
5 )
6 generated_sql = extract_sql(response.choices[0].message.content)

```

3.1.4 Layer 4: Query Execution

The execution layer implements comprehensive safety measures to securely execute generated SQL queries against target databases while maintaining data integrity and system stability. The primary safety mechanism involves enforcing read-only mode through the SQLite pragma directive `PRAGMA query_only = ON`, which prevents any data modification operations and ensures that query execution cannot alter database contents regardless of the specific SQL statements processed.

Connection isolation represents another critical safety feature, where each query utilizes a dedicated database connection that operates independently of other concurrent operations. This isolation prevents interference between simultaneous queries and ensures that connection-specific settings or temporary modifications do not affect other system operations. Automatic cleanup procedures guarantee that database connections are properly closed after query execution, preventing resource leaks and maintaining system performance under high-volume operation scenarios.

The implementation follows a systematic approach to error handling and result processing:

Listing 3.3: SQL Execution with Safety Measures

```

1 def run_sql(db_id: str, sql: str):
2     try:
3         if not is_select_sql(sql):
4             return "SQL ERROR: only SELECT/CTE queries allowed"
5         conn = sqlite3.connect(db_path_for(db_id))
6         conn.execute("PRAGMA query_only = ON;")
7         df = pd.read_sql_query(sql, conn)
8         conn.close()
9         return df
10    except Exception as e:
11        return f"SQL ERROR: {e}"

```

This implementation validates query types before execution, establishes secure database

connections with read-only constraints, processes queries using `pandas` for consistent result formatting, and captures comprehensive error information when execution failures occur. The systematic error handling ensures that all failures are properly categorized and formatted for subsequent analysis by the repair mechanism.

3.1.5 Layer 5: Validation and Correction (Key Innovation)

The validation and correction layer represents our primary contribution to the six-layer text-to-SQL architecture, implementing an intelligent repair mechanism that automatically detects and corrects SQL generation failures. Building on previous research by Shen et al. (2025) in MapleRepair [1] and Wang et al. (2024) in DAC decomposed correction approaches [2], our system extends these concepts through multi-dimensional error detection and iterative repair processes specifically designed for large language model-based text-to-SQL systems.

Error Detection and Classification Framework

Our error detection system operates on four distinct error categories, expanding beyond the binary success-failure paradigm identified in previous text-to-SQL research [5]. The system addresses: (1) SQL execution errors including syntax violations and constraint violations that prevent query execution, (2) Result mismatches where queries execute successfully but produce incorrect output, (3) Schema violations involving non-existent table or column references, and (4) Semantic logic errors such as incorrect aggregation or missing operations.

This implementation directly addresses the schema linking challenges identified by Deng et al. (2021) in STRUG [strug2021], where text-table alignment represents a critical bottleneck in text-to-SQL accuracy. The error detection framework operates by first attempting SQL execution, then checking for syntax errors if execution fails, followed by schema validation, and finally result comparison when gold standard answers are available.

Question Intent Analysis Implementation

The question intent analysis system implements sophisticated linguistic pattern recognition to understand user requirements and generate targeted repair hints. Our implementation analyzes natural language questions to identify specific SQL operations required through keyword pattern matching and contextual analysis.

Listing 3.4: Question Intent Analysis from Our Implementation

```
1 def analyze_question_intent(question: str, db_id: str) -> str:
2     """Generate intelligent repair hints based on question
3         analysis"""
4     question_lower = question.lower()
5     hints = []
6     # Count-related patterns
```



```

7      if any(word in question_lower for word in
8              ['how many', 'count', 'number of', 'total']):
9          hints.append("- This question asks for COUNT - use COUNT
10                        (*)")
11
12      # Average patterns
13      if any(word in question_lower for word in
14              ['average', 'mean', 'avg']):
15          hints.append("- This question asks for AVERAGE - use AVG(
16                        column)")
17
18      # Maximum patterns
19      if any(word in question_lower for word in
20              ['highest', 'maximum', 'max', 'largest', 'most']):
21          hints.append("- This question asks for MAXIMUM - use MAX(
22                        column)")
23
24      # Grouping patterns
25      if any(word in question_lower for word in
26              ['each', 'every', 'per', 'by', 'for each']):
27          hints.append("- This question asks for grouping - use
28                        GROUP BY")
29
30      return "\n".join(hints) if hints else "- Analyze question
31      structure"

```

The question analysis pipeline processes natural language input through pattern detection modules that identify COUNT operations, aggregation functions like AVG and MAX, grouping requirements, and potential JOIN operations. For example, the question "How many students are in each class?" triggers both COUNT and GROUP BY pattern detection, generating targeted repair hints for both operations.

Iterative Repair Process Implementation

The repair mechanism implements up to three correction attempts, following our system architecture design. Each iteration receives increasingly specific guidance based on previous failure analysis, implementing concepts from MapleRepair's systematic error correction framework.

Listing 3.5: Iterative Repair Process from Our System

```

1 def predict_with_repairs(question: str, db_id: str, gold_result=
  None):
2     """Enhanced prediction function with intelligent repair
      mechanism"""
3     # Initial SQL generation
4     sql = llm_generate_sql(question, db_id)
5     out = run_sql(db_id, sql)
6     attempts = 0
7     repair_reasons = []
8
9     while attempts < MAX_REPAIRS:
10         need_repair = False
11         error_msg = None
12
13         # Check for SQL execution errors
14         if isinstance(out, str) and out.startswith("SQL ERROR"):
15             need_repair = True
16             error_msg = out
17             repair_reasons.append("SQL Error")
18
19         # Check for wrong results (if gold result is provided)
20         elif gold_result is not None and not isinstance(out, str)
21             :
22             exact, _, _, _ = compare(gold_result, out)
23             if not exact:
24                 need_repair = True
25                 error_msg = generate_error_message(gold_result,
26                                                         out)
27                 repair_reasons.append("Wrong Result")
28
29         if not need_repair:
30             break
31
32         # Attempt repair
33         sql = llm_repair_sql(question, db_id, sql, error_msg,
34                                 gold_result, out)
35         out = run_sql(db_id, sql)
36         attempts += 1
37
38     return sql, out, attempts, repair_reasons

```

The iterative repair process begins with initial SQL generation, followed by execution attempt and error detection. If repair is needed, the system generates corrected SQL using error analysis and question intent hints, then repeats the process for up to three attempts. This approach enables progressive enhancement where each repair attempt receives more specific guidance based on accumulated failure context.

Experimental Results and Performance Analysis

Our repair system was evaluated across multiple LLM architectures using the Spider dataset [3], demonstrating varying effectiveness depending on the underlying model capabilities. Table 3.1 presents the comprehensive results across three different models.

Table 3.1: Repair System Performance Across Different LLM Models

Model	Original Question Accuracy	Improved Question Accuracy	Repair Attempts	Success Rate
GPT-4o	866/1034 (83.8%)	846/1034 (81.8%)	332	49.4%
GPT-4o-mini	830/1034 (80.3%)	802/1034 (77.6%)	358	43.0%
LLaMA Coder	519/1034 (50.2%)	478/1034 (46.2%)	620	16.9%
Qwen Coder	465/1034 (45.0%)	445/1034 (43.0%)	598	4.8%

These results demonstrate that repair effectiveness correlates strongly with the underlying LLM’s SQL generation capabilities. Higher-performing models like GPT-4o show substantial repair success rates of 49.4%, while weaker models like Qwen Coder show limited repair effectiveness at only 4.8%. This indicates that the repair mechanism requires a baseline level of SQL understanding to function effectively.

Notably, the slight decrease in overall accuracy for some models (83.8% to 81.8% for GPT-4o) reveals an important limitation: the repair process may introduce additional complexity that current mechanisms cannot fully address. This counterintuitive finding suggests that repair systems cannot simply be applied universally and may require more sophisticated activation criteria.

Integration with Six-Layer Architecture

The validation layer seamlessly integrates with our proposed six-layer architecture through well-defined interfaces. The integration maintains clear data flow: Layer 1 processes user input, Layer 2 constructs schema-aware prompts, Layer 3 generates initial SQL via LLM APIs, Layer 4 attempts database execution, Layer 5 (our contribution) performs validation and correction with feedback to Layer 3 if repair is needed, and Layer 6 formats final output presentation.

Listing 3.6: Complete Six-Layer Pipeline Integration

```

1 def complete_text_to_sql_pipeline(question: str, db_id: str) ->
  dict:
2     """Complete six-layer pipeline with validation and correction
        """
3
4     # Layer 1: Input processing
5     processed_question = preprocess_input(question)
6
7     # Layer 2: Prompt engineering
8     schema_prompt = build_schema_prompt(processed_question, db_id
9         )
10
11     # Layer 3: LLM generation
12     initial_sql = llm_generate_sql(schema_prompt, db_id)
13
14     # Layer 4: Execution attempt
15     execution_result = run_sql(db_id, initial_sql)
16
17     # Layer 5: Validation and correction (OUR CONTRIBUTION)
18     if requires_repair(execution_result):
19         corrected_sql, final_result, attempts, reasons =
20             predict_with_repairs(
21                 question, db_id
22             )
23         return {
24             'sql': corrected_sql,
25             'result': final_result,
26             'repair_attempts': attempts,
27             'repair_reasons': reasons
28         }
29
30     # Layer 6: Output formatting
31     return format_final_output(initial_sql, execution_result)

```

The repair system receives input from the SQL generation layer, accesses schema information from the database layer, and provides corrected outputs to the execution layer without disrupting the overall system architecture. This design maintains separation of concerns while enabling comprehensive error correction capabilities.

Key Contributions and Limitations Analysis

Our validation layer contributes three key innovations to text-to-SQL systems: multi-dimensional error detection beyond simple execution failures, intent-aware repair guidance using linguistic pattern analysis, and iterative correction process with progressive enhancement.

However, our experimental results reveal important limitations. The repair mechanism’s effectiveness depends heavily on the underlying LLM’s capabilities, with success rates ranging from 4.8% (Qwen Coder) to 49.4% (GPT-4o). This suggests that repair systems cannot compensate for fundamental deficiencies in SQL generation capabilities.

Additionally, the slight decrease in performance for some models indicates that the repair process may introduce additional complexity that current repair mechanisms cannot fully address. This highlights an important area for future research in developing more sophisticated repair activation criteria and correction strategies.

Connection to Previous Research

Our approach builds directly on several foundational works. STRUG [strug2021] identified schema linking challenges that we address through our error detection framework. MapleRepair [1] provided ICL error correction concepts that we extend to multi-dimensional repair strategies. DAC [2] introduced decomposed correction approaches that we implement within our iterative repair process. The survey findings by Zhu et al. [5] identified query complexity challenges that we address through targeted repair guidance.

This validation layer represents a significant advancement in text-to-SQL robustness, providing a systematic approach to error recovery that enhances overall system reliability while maintaining integration with modern LLM-based architectures.

3.2 Dataset Description

Our experimental evaluation utilizes the Spider 1.0 dataset [3] for text-to-SQL evaluation. Based on our data analysis, the Spider dataset contains the following verified components with their complete column structures.

3.2.1 Spider Dataset Structure

Table 3.3 presents the actual structure of the Spider dataset components as confirmed through our data inspection.

Table 3.2: Spider Dataset Complete Structure with Column Details

Component	Entries	Columns	Column Names
Training Set	7,000	7	db_id, query, query_toks, query_toks_no_value, question, question_toks, sql
Development Set	1,034	7	db_id, query, query_toks, query_toks_no_value, question, question_toks, sql
Other Set	1,659	7	db_id, query, query_toks, query_toks_no_value, question, question_toks, sql
Database Tables	166	8	column_names, column_names_original, column_types, db_id, foreign_keys, primary_keys, table_names, table_names_original

3.2.2 Development Set Details

For our experimental evaluation, we utilize the development set containing 1,034 entries. Each entry contains complete data across all 7 columns:

- **db_id**: Database identifier (1,034 non-null object entries)
- **query**: SQL query string (1,034 non-null object entries)
- **query_toks**: Tokenized query representation (1,034 non-null object entries)
- **query_toks_no_value**: Tokenized query without values (1,034 non-null object entries)
- **question**: Natural language question (1,034 non-null object entries)

- **question_toks**: Tokenized question representation (1,034 non-null object entries)
- **sql**: SQL structure information (1,034 non-null object entries)

3.2.3 Database Schema Information

The database schema component contains 166 entries with 8 columns providing complete structural information:

- **column_names**: Column name representations (166 non-null object entries)
- **column_names_original**: Original column names (166 non-null object entries)
- **column_types**: Data type specifications (166 non-null object entries)
- **db_id**: Database identifier (166 non-null object entries)
- **foreign_keys**: Foreign key relationships (166 non-null object entries)
- **primary_keys**: Primary key specifications (166 non-null object entries)
- **table_names**: Table name representations (166 non-null object entries)
- **table_names_original**: Original table names (166 non-null object entries)

All dataset components contain complete data with no null values, ensuring reliable evaluation across the entire dataset.

Table 3.3: Spider Dataset Structure and Components

Component	Entries	Columns	Description
Training Set	7,000	7	Natural language questions with corresponding SQL queries for model training
Development Set	1,034	7	Evaluation queries used for testing and validation (our primary evaluation set)
Other Queries	1,659	7	Additional query examples for extended evaluation scenarios
Database Schema	166	8	Complete database schema information including table structures and relationships

3.2.4 Dataset Characteristics

The Spider dataset encompasses several key characteristics that make it particularly suitable for evaluating repair-enhanced text-to-SQL systems:

Cross-Domain Coverage: The dataset spans 166 different databases across diverse domains including academic institutions, business operations, entertainment, sports, and government applications. This cross-domain structure ensures that evaluation results reflect system performance across varied real-world scenarios rather than domain-specific optimization.

Query Complexity Distribution: The 1,034 development set queries range from simple single-table operations to complex multi-table joins involving nested subqueries, aggregation functions, and advanced SQL constructs. This complexity distribution enables comprehensive assessment of repair mechanism effectiveness across different query difficulty levels.

Zero Database Overlap: The evaluation maintains strict separation between training and testing databases, ensuring that systems cannot memorize specific database schemas and must demonstrate true generalization capabilities across unseen database structures.

3.2.5 Evaluation Methodology

For our experimental evaluation, we primarily utilize the development set containing 1,034 questions as our main evaluation corpus. Each entry in the development set contains the following essential components:

- **Natural Language Question:** User queries expressed in natural language that represent realistic information needs
- **Database Identifier:** Specific database schema against which the query should be executed
- **Gold Standard SQL:** Expert-annotated SQL queries that produce the correct results for each natural language question
- **Tokenized Representations:** Pre-processed versions of both questions and SQL queries for analysis purposes

The strict evaluation protocol requires generated SQL queries to produce results identical to the gold standard queries, ensuring precise measurement of system accuracy and enabling reliable assessment of repair mechanism effectiveness. This exact match evaluation criterion

provides unambiguous success metrics while maintaining the rigor necessary for meaningful performance comparisons across different system configurations and model architectures.

3.3 Results and Analysis

3.3.1 Overall Performance Assessment

Our comprehensive evaluation on the Spider development dataset containing 1,034 complex cross-domain text-to-SQL samples reveals significant findings about LLM performance in structured data querying tasks. The evaluation demonstrates substantial accuracy differences across models and provides critical insights into the effectiveness of automated repair mechanisms.

Model Performance Comparison

GPT-4o achieved the highest baseline performance with 866/1034 (83.8%) exact match accuracy on original questions, demonstrating strong capability in converting natural language to executable SQL queries. The model successfully interpreted database schemas, understood natural language questions, and generated syntactically and semantically correct SQL in most cases. GPT-4o-mini achieved 830/1034 (80.3%) accuracy, showing a 3.5 percentage point performance gap that reflects the benefits of the larger model’s additional parameters and training.

In contrast, Qwen Coder demonstrated significantly lower performance at 465/1034 (45.0%) accuracy, indicating substantial limitations in SQL generation capabilities. This 38.8 percentage point gap between GPT-4o and Qwen Coder highlights the critical importance of model selection in text-to-SQL applications and suggests that not all LLMs are equally capable of structured data querying tasks.

Question Improvement Impact Analysis

A counterintuitive finding emerged regarding the impact of question improvements on model performance. GPT-4o’s accuracy decreased from 83.8% to 81.8% when processing improved questions, representing a loss of 20 correctly answered questions. GPT-4o-mini showed a larger decline from 80.3% to 77.6%, losing 28 correct answers. Qwen Coder maintained similar poor performance with a decrease from 45.0% to 43.0%.

This performance degradation contradicts expectations that clearer, more explicit questions would improve accuracy. The consistent decline across all models suggests that question improvements may have introduced linguistic complexity or patterns less familiar to the models’ training data. This finding has significant implications for user interface design in database querying systems.

Table 3.4: Model Performance Comparison on Original vs Improved Questions

Model	Original Questions	Improved Questions	Difference
GPT-4o	866/1034 (83.8%)	846/1034 (81.8%)	-20 (-1.9%)
GPT-4o-mini	830/1034 (80.3%)	802/1034 (77.6%)	-28 (-2.7%)
LLaMA Coder	519/1034 (50.2%)	478/1034 (46.2%)	-41 (-4.0%)
Qwen Coder	465/1034 (45.0%)	445/1034 (43.0%)	-20 (-1.9%)

3.3.2 Repair Mechanism Performance and Effectiveness

The intelligent repair system represents our core contribution, implementing iterative correction mechanisms that attempt to fix initially failed SQL queries through targeted analysis and regeneration. The repair system demonstrated varying effectiveness across models, with success rates strongly correlating with underlying model capabilities.

Repair Success Rate Analysis

GPT-4o achieved the highest repair success rate at 49.4% (164/332 successful repairs) on original questions, indicating that approximately half of initially failed queries could be corrected through automated repair. The repair attempts increased to 358 for improved questions with a slightly lower success rate of 47.5% (170/358). This demonstrates that while improved questions caused more initial failures, they also created errors that were more challenging for the repair mechanism to correct.

GPT-4o-mini showed lower repair effectiveness with 43.0% success (154/358) on original questions and 42.7% success (173/405) on improved questions. The model required repairs on a higher percentage of questions (34.6% vs 32.1% for GPT-4o), indicating greater initial failure rates but similar repair difficulty levels.

Qwen Coder demonstrated severely limited repair effectiveness with only 4.8% success (29/598) on original questions and 3.4% success (21/610) on improved questions. The model required repair attempts on 57.8% of original questions and 59.0% of improved questions, indicating fundamental SQL generation deficiencies that the repair mechanism could not address.

Error Pattern Classification and Repair Effectiveness

Analysis of repair attempts revealed three primary error categories with varying correction success rates. Schema-related errors including incorrect table or column name references showed the highest repair success rates, particularly for GPT-4o and GPT-4o-mini. These errors often resulted from case sensitivity issues or minor spelling variations that could be corrected through targeted schema information in repair prompts.

Table 3.5: Repair Mechanism Effectiveness Across Models and Question Types

Model	Question Type	Repair Attempts	Successful	Success Rate
GPT-4o	Original	332	164	49.4%
	Improved	358	170	47.5%
GPT-4o-mini	Original	358	154	43.0%
	Improved	405	173	42.7%
LLaMA Coder	Original	620	105	16.9%
	Improved	642	86	13.4%
Qwen Coder	Original	598	29	4.8%
	Improved	610	21	3.4%

Syntax errors involving malformed SQL structure, incorrect parentheses placement, or missing keywords demonstrated moderate repair success rates. The repair mechanism’s question intent analysis proved effective in identifying missing aggregation functions or incorrect SQL operations, enabling successful corrections in many cases.

Semantic reasoning errors represented the most challenging category, involving fundamental misunderstandings of question intent or logical relationships between database entities. These errors often produced syntactically correct queries that executed successfully but returned incorrect results due to wrong aggregation functions, missing filtering conditions, or improper multi-table relationships. The repair mechanism showed limited success with these errors across all models.

Table 3.6: Complete Performance Analysis of Repair Mechanisms Across Models

Model	Original Questions			Improved Questions		
	Without Repair	With Repair	Improve-ment	Without Repair	With Repair	Improve-ment
GPT-4o	67.9%	83.8%	+15.9%	65.4%	81.8%	+16.4%
GPT-4o-mini	65.4%	80.3%	+14.9%	60.8%	77.6%	+16.7%
LLaMA Coder	40.0%	50.2%	+10.2%	37.9%	46.2%	+8.3%
Qwen Coder	42.2%	45.0%	+2.8%	41.0%	43.0%	+2.0%

3.3.3 Statistical Significance and Reliability

The evaluation results demonstrate statistical significance due to the substantial sample size of 1,034 questions and consistent patterns observed across different model configurations. The performance differences between models represent meaningful distinctions that exceed statistical noise, particularly the 36-44 additional correct answers separating GPT-4o from GPT-4o-mini across question types.

The repair mechanism’s effectiveness, while moderate for stronger models, demonstrates statistical reliability across large numbers of repair attempts. With total repair attempts

ranging from 332 to 610 depending on configuration, the observed success rates provide confidence in the reliability of these findings and suggest stable characteristics rather than experimental artifacts.

3.4 Discussion and Implications

3.4.1 Understanding the Question Improvement Paradox

The counterintuitive finding that improved questions resulted in decreased performance across all models challenges assumptions about the relationship between question clarity and LLM performance in text-to-SQL tasks. This phenomenon suggests several important implications for system design and deployment.

The question improvement process may have inadvertently introduced linguistic complexity or patterns less familiar to the models’ training data. While improved questions appear clearer to human readers, they might contain syntactic patterns or vocabulary choices that deviate from the text-to-SQL examples used in model training. Since LLMs rely heavily on pattern matching and statistical associations learned from training data, questions that deviate from familiar patterns may actually impair performance despite seeming more explicit.

Another explanation relates to potential semantic ambiguity introduced through the improvement process. Original Spider questions, despite appearing less clear, may have been formulated to correspond more directly to database structures and SQL logic. Question improvements aimed at natural language clarity might introduce multiple interpretations or require more sophisticated reasoning to map to correct SQL operations.

This finding has significant implications for user interface design in database querying systems, suggesting that more natural-sounding questions may not always produce better results. System designers should consider training users on question formulation patterns that align with model capabilities rather than assuming that clearer natural language necessarily improves performance.

3.4.2 Repair Mechanism Contributions and Limitations

The repair mechanism demonstrates both significant potential and important limitations in automated error correction for text-to-SQL systems. The ability to successfully correct 43-49% of initially failed queries for strong models like GPT-4o represents substantial practical value, effectively recovering 154-173 additional correct answers per 1,034 questions.

The repair system’s effectiveness varied significantly by error type, achieving highest success rates for schema-related errors and syntax mistakes. The system’s ability to provide targeted hints based on question analysis, combined with complete schema information and specific error feedback, enabled models to correct many systematic mistakes related to database structure references.

However, the repair mechanism’s moderate success rate reveals important limitations in current approaches to automated error correction. The failure to correct 50-57% of initially wrong queries for strong models, and over 95% for weak models like Qwen Coder, indicates that many errors involve deeper semantic misunderstandings that cannot be addressed through additional prompting alone.

The repair mechanism’s severely limited effectiveness on Qwen Coder (4.8% success rate) demonstrates that repair systems cannot compensate for fundamental deficiencies in SQL generation capabilities. This finding suggests that repair mechanisms require a baseline level of model competence to function effectively and cannot serve as universal solutions for poor text-to-SQL performance.

3.4.3 Implications for Text-to-SQL System Design

These findings provide several important guidelines for text-to-SQL system design and deployment. First, model selection critically impacts both baseline performance and repair mechanism effectiveness. Organizations implementing text-to-SQL systems should prioritize high-capability models like GPT-4o over weaker alternatives, as the performance differences are substantial and repair mechanisms cannot bridge fundamental capability gaps.

Second, question preprocessing and user training may be more important than initially assumed. The question improvement paradox suggests that systems should guide users toward question formulation patterns that align with model training rather than encouraging more natural language expressions that may actually impair performance.

Third, repair mechanisms provide meaningful value for high-capability models but should not be expected to solve fundamental model limitations. The 43-49% repair success rates for strong models justify the implementation complexity, while the poor repair performance on weak models suggests that repair systems are not viable solutions for fundamentally inadequate text-to-SQL capabilities.

Finally, the consistent performance patterns across different question types and models suggest that these findings are likely to generalize to other text-to-SQL applications and datasets, providing reliable guidance for system design decisions in various organizational contexts.

Chapter 4

Experiments and Results

This chapter presents the experimental evaluation of our repair-enhanced text-to-SQL system. We report comprehensive results across multiple language models, analyze repair mechanism effectiveness, and examine the counterintuitive effects of question reformulation on system performance.

4.1 Experimental Setup

4.1.1 Dataset and Evaluation Framework

Our experimental evaluation utilized the Spider development dataset [3], containing 1,034 complex cross-domain text-to-SQL samples across 20 different databases. Spider represents the gold standard for text-to-SQL evaluation, featuring challenging queries with multi-table joins, nested subqueries, and sophisticated aggregation operations that comprehensively test system capabilities.

The evaluation employed exact match accuracy as the primary metric, requiring generated SQL queries to produce results identical to gold standard queries in both structure and content. This strict evaluation criterion ensures that our system not only generates syntactically correct SQL but also captures the precise semantic intent of natural language questions.

4.1.2 Model Configuration and Infrastructure

We evaluated three distinct language models to assess performance across different capability levels. GPT-4o and GPT-4o-mini were accessed through OpenAI’s official API using authenticated access keys, with all computations performed on OpenAI’s cloud

infrastructure. Qwen Coder was executed locally within Google Colab’s computational environment to provide comparison with open-source alternatives.

All models were configured with temperature=0 to ensure deterministic output generation and reproducible results. API timeout was set to 30 seconds per request with a 0.2-second delay between consecutive requests to respect rate limiting guidelines. The repair system maintained consistent configuration across all evaluations with a maximum of three repair attempts per query, incorporating our question intent analysis system for targeted correction guidance.

4.2 Main Results

4.2.1 Overall Performance Analysis

Table 4.1 presents our primary findings across the tested language models on the complete Spider development dataset.

Table 4.1: Text-to-SQL Performance on Spider Development Set (1,034 samples)

Model	Original Questions	Improved Questions	Performance Change
GPT-4o	866/1034 (83.8%)	846/1034 (81.8%)	-20 (-1.9%)
GPT-4o-mini	830/1034 (80.3%)	802/1034 (77.6%)	-28 (-2.7%)
LLaMA Coder	519/1034 (50.2%)	478/1034 (46.2%)	-41 (-4.0%)
Qwen Coder	465/1034 (45.0%)	445/1034 (43.0%)	-20 (-1.9%)

GPT-4o achieved the highest baseline performance with 83.8% exact match accuracy on original questions, correctly generating 866 out of 1,034 SQL queries. This performance demonstrates strong capability in understanding database schemas, interpreting natural language questions, and generating syntactically and semantically correct SQL queries for complex cross-domain scenarios.

GPT-4o-mini achieved competitive performance with 80.3% exact match accuracy, correctly handling 830 questions. The 3.5 percentage point difference between models indicates meaningful advantages for the larger model in handling complex multi-table joins and sophisticated aggregation operations.

Qwen Coder demonstrated significantly lower performance at 45.0% accuracy, correctly answering 465 questions. This 38.8 percentage point gap compared to GPT-4o highlights substantial differences in SQL generation capabilities across model architectures and training approaches.

Question Improvement Paradox

Our most significant finding relates to the counterintuitive performance degradation observed when processing "improved" questions rather than original Spider formulations. GPT-4o’s performance decreased from 83.8% to 81.8%, losing 20 correct answers. GPT-4o-mini showed larger decline from 80.3% to 77.6%, losing 28 correct answers. Even Qwen Coder experienced degradation from 45.0% to 43.0%.

This consistent performance decline across all models contradicts expectations that clearer, more explicit question formulations should improve system performance. The finding suggests that question reformulation may introduce linguistic patterns less familiar to model training data or create semantic ambiguities that interfere with SQL mapping processes.

4.2.2 Repair Mechanism Performance Analysis

Table 4.2 provides comprehensive analysis of our repair system effectiveness across different model and question configurations.

Table 4.2: Repair System Performance Analysis

Configuration	Total Attempts	Successful	Failed	Success Rate
GPT-4o Original	332	164	168	49.4%
GPT-4o Improved	358	170	188	47.5%
GPT-4o-mini Original	358	154	204	43.0%
GPT-4o-mini Improved	405	173	232	42.7%
LLaMA Coder Original	620	105	515	16.9%
LLaMA Coder Improved	642	86	556	13.4%
Qwen Coder Original	598	29	569	4.8%
Qwen Coder Improved	610	21	589	3.4%

The repair mechanism demonstrated effectiveness that strongly correlates with underlying model capabilities. GPT-4o achieved the highest repair success rate of 49.4% on original questions, successfully correcting 164 out of 332 repair attempts. This indicates that approximately half of initially failed queries could be recovered through automated repair, representing substantial practical value.

GPT-4o-mini showed lower but still meaningful repair effectiveness with 43.0% success on original questions and 42.7% on improved questions. The model required repair attempts on higher percentages of questions, indicating greater initial failure rates but similar repair difficulty levels.

Qwen Coder demonstrated severely limited repair effectiveness with only 4.8% success on original questions and 3.4% on improved questions. The model required repair attempts

on 57.8% of original questions and 59.0% of improved questions, indicating fundamental SQL generation deficiencies that the repair mechanism could not address.

Repair Demand Analysis

The increased repair demand for improved questions provides additional evidence of their complexity. GPT-4o required 7.8% more repair attempts for improved questions (358 vs 332), while GPT-4o-mini needed 13.1% more attempts (405 vs 358). Qwen Coder showed minimal change in repair demand, likely due to its already high failure rate.

The repair success rates for improved questions remained similar to or slightly lower than those for original questions, suggesting that errors introduced by question reformulation involve fundamental misalignments rather than superficial mistakes easily correctable through additional prompting.

4.3 Error Analysis and System Limitations

4.3.1 Common Error Categories

Analysis of repair attempts reveals three primary error categories with varying correction success rates. Schema-related errors including incorrect table or column name references showed highest repair success rates for GPT-4o and GPT-4o-mini. These errors often resulted from case sensitivity issues or minor spelling variations correctable through targeted schema information.

Syntax errors involving malformed SQL structure, incorrect parentheses placement, or missing keywords demonstrated moderate repair success rates. The repair mechanism’s question intent analysis proved effective in identifying missing aggregation functions or incorrect SQL operations.

Semantic reasoning errors represented the most challenging category, involving fundamental misunderstandings of question intent or logical relationships between database entities. These errors often produced syntactically correct queries that executed successfully but returned incorrect results due to wrong aggregation functions, missing filtering conditions, or improper multi-table relationships.

4.3.2 Model-Specific Limitations

GPT-4o and GPT-4o-mini showed similar error patterns but different frequencies, with the smaller model requiring more frequent repairs. Both models handled schema-related

and syntax errors effectively but struggled with complex semantic reasoning.

Qwen Coder’s poor repair success rate (4.8%) indicates fundamental limitations in SQL generation capabilities that cannot be addressed through prompt-based correction. The model’s high repair demand (57.8% of questions) combined with low success rate suggests that repair mechanisms require baseline model competence to function effectively.

4.4 Statistical Significance and Reliability

The results demonstrate statistical significance due to the substantial sample size of 1,034 questions and consistent patterns across model configurations. Performance differences between models represent meaningful distinctions exceeding random variation, particularly the 36-44 additional correct answers separating GPT-4o from GPT-4o-mini.

The repair mechanism’s effectiveness demonstrates statistical reliability across large numbers of repair attempts. With total attempts ranging from 332 to 610 depending on configuration, observed success rates provide confidence in finding reliability and suggest stable characteristics rather than experimental artifacts.

The consistent question improvement paradox across all three models, with decreases of 20-28 correct answers for strong models and similar patterns for weak models, indicates systematic effects rather than statistical noise.

4.5 Discussion and Implications

4.5.1 Key Findings Summary

This experimental evaluation reveals several critical insights for text-to-SQL system design. First, model selection critically impacts both baseline performance and repair mechanism effectiveness, with GPT-4o demonstrating clear advantages over alternatives. Second, the repair mechanism provides meaningful value for high-capability models (43-49% success rates) but cannot compensate for fundamental model limitations (4.8% success for weak models).

Third, the question improvement paradox challenges assumptions about optimal question formulation, suggesting that more natural-sounding questions may actually impair performance in current text-to-SQL systems. This finding has important implications for user interface design and user training approaches.

4.5.2 Practical Implications

These findings provide actionable guidance for text-to-SQL system implementation. Organizations should prioritize high-capability models like GPT-4o over weaker alternatives, as performance differences are substantial and repair mechanisms cannot bridge fundamental capability gaps.

Question preprocessing and user training may be more important than initially assumed. Systems should guide users toward question formulation patterns that align with model training rather than encouraging more natural language expressions that may impair performance.

Repair mechanisms justify implementation complexity for strong models but should not be expected to solve fundamental model limitations. The 43-49% repair success rates for strong models provide meaningful reliability improvements, while poor repair performance on weak models confirms that repair systems are not universal solutions for inadequate text-to-SQL capabilities.

4.5.3 Future Research Directions

The question sensitivity findings warrant dedicated research into optimal question formulation strategies and development of more robust models that handle natural language variations effectively. Investigation of semantic equivalence checking and execution accuracy metrics could provide more nuanced evaluation frameworks that better capture system capabilities.

Future research should focus on developing more sophisticated error analysis and correction strategies, particularly for complex semantic reasoning errors that prove resistant to current repair approaches. Additionally, understanding the mechanisms behind the question improvement paradox could lead to more robust text-to-SQL architectures that maintain performance across natural language variations.

Chapter 5

Discussion

This chapter provides a comprehensive analysis of our experimental results from testing the intelligent text-to-SQL system with automated repair mechanisms across GPT-4o and GPT-4o-mini language models. We interpret findings from the Spider development dataset evaluation, discuss implications of our repair system’s performance, acknowledge limitations, and address the counterintuitive question sensitivity findings.

5.1 Interpretation of Results

5.1.1 Baseline Performance Achievement

Our evaluation on the Spider development dataset containing 1,034 samples demonstrates that current large language models achieve substantial baseline performance in text-to-SQL generation tasks. GPT-4o achieved 83.8% exact match accuracy on original questions, correctly generating 866 out of 1,034 SQL queries that produced identical results to the gold standard. This performance level represents a significant advancement in the practical applicability of language models for database querying tasks, approaching levels suitable for real-world deployment scenarios.

GPT-4o-mini demonstrated competitive performance with 80.3% exact match accuracy, correctly handling 830 out of 1,034 questions. The 3.5 percentage point difference between the models indicates that the larger GPT-4o provides meaningful advantages in handling complex multi-table joins, sophisticated aggregation operations, and nuanced natural language interpretations. However, both models achieve performance levels that exceed the typical 75-80% threshold considered viable for production applications.

The strong baseline performance across both models validates the substantial progress made in language model capabilities for structured data tasks. These results demonstrate

that modern LLMs have developed sophisticated understanding of both natural language semantics and SQL syntax, enabling them to successfully navigate the complex mapping between linguistic expressions and database operations across diverse domains and query types.

5.1.2 Repair Mechanism Effectiveness

The intelligent repair system demonstrated moderate but meaningful effectiveness across both tested models, with success rates ranging from 42.7% to 49.4% depending on configuration. For GPT-4o processing original questions, the repair system attempted corrections on 332 out of 1,034 queries, achieving success in 164 cases for a 49.4% repair success rate. This performance indicates that approximately half of initially failed queries could be recovered through automated repair processes, representing substantial value for system reliability and user experience.

When processing improved questions, GPT-4o required repair attempts on 358 queries, representing a 7.8% increase in repair demand. Despite this increased intervention requirement, the system achieved 170 successful repairs with a slightly lower 47.5% success rate. This pattern suggests that improved questions not only cause more initial failures but also create errors that are more resistant to correction through additional prompting and guidance.

GPT-4o-mini demonstrated consistently lower repair success rates across both question types, achieving 43.0% success on original questions and 42.7% success on improved questions. The model required repair attempts on 358 original questions and 405 improved questions, representing higher intervention rates compared to GPT-4o. The total repair demand of 39% for improved questions compared to 35% for original questions provides additional evidence that question reformulation systematically increases generation difficulty.

The repair mechanism successfully addressed many common error patterns, including schema-related issues involving incorrect table or column names, syntax errors with malformed SQL structure, and logical errors in aggregation function selection. However, the failure to correct 50-57% of initially wrong queries indicates that many errors involve deeper semantic misunderstandings or complex reasoning failures that cannot be addressed through current repair approaches.

5.1.3 Question Sensitivity Discovery

The most significant and counterintuitive finding from our evaluation concerns the consistent performance degradation observed when processing "improved" questions compared to original Spider dataset formulations. This unexpected result challenges conventional assumptions about the relationship between question clarity and system performance in text-to-SQL tasks. Both models demonstrated systematic performance degradation on improved questions, with GPT-4o showing a 1.9 percentage point decrease and GPT-4o-mini exhibiting a 2.7 percentage point reduction.

This pattern suggests several possible explanations for the phenomenon. The question improvement process may have inadvertently introduced linguistic complexity or ambiguity that interferes with the models' ability to map natural language to SQL structures. While improved questions may appear clearer to human readers, they might contain syntactic patterns or vocabulary choices that are less common in the training data used to develop these language models. Since LLMs rely heavily on pattern matching and statistical associations learned from training data, questions that deviate from familiar patterns may actually be more difficult to process correctly.

Another hypothesis relates to potential semantic ambiguity introduction through the improvement process. The original Spider questions, despite potentially appearing less clear to human readers, may have been formulated in ways that more directly correspond to underlying database structure and SQL logic. When questions are "improved" to be more natural or conversational, they might introduce multiple possible interpretations or require more sophisticated reasoning to map to correct SQL operations.

The question sensitivity findings have important implications for system design and user interface development. The results suggest that more natural-sounding questions may not always produce better results in current text-to-SQL systems, indicating that user training or query suggestion systems might need to guide users toward question formulations that align with model training patterns rather than encouraging natural language elaboration.

5.2 Error Analysis and System Performance

5.2.1 Common Error Categories and Patterns

Our analysis of repair attempts reveals several distinct categories of errors that characterize the challenges faced by current language models in text-to-SQL generation. Schema-related errors represent a significant portion of failures, typically involving incorrect table or column name references often due to case sensitivity issues, minor spelling variations,

or confusion between similar column names across different tables. These errors were among the most successfully addressed by the repair mechanism, as targeted hints about exact schema matching often enabled models to correct these mistakes.

Syntax errors constitute another major category, including malformed SQL queries with incorrect parentheses placement, missing keywords such as `SELECT` or `FROM`, and improper `JOIN` syntax. The repair mechanism showed moderate success with these errors, particularly when the underlying logical structure of the query was correct but surface-level syntax contained mistakes. However, more complex syntax errors involving nested subqueries or sophisticated conditional logic proved more resistant to correction.

Semantic reasoning errors represent the most challenging category for the repair system, involving fundamental misunderstandings of question intent or logical relationships between database entities. These errors often manifest as queries that are syntactically correct and execute successfully but produce incorrect results due to wrong aggregation functions, missing or incorrect filtering conditions, or improper handling of multi-table relationships. The repair mechanism’s limited success with these errors highlights the fundamental challenge of providing automated feedback that can guide models toward correct semantic interpretations of complex natural language queries.

5.2.2 Repair Success Patterns

The analysis of repair success patterns reveals important insights about the types of corrections that are feasible through automated mechanisms versus those that require more sophisticated intervention. Simple corrections involving schema name adjustments or basic syntax fixes demonstrate high success rates, indicating that many LLM errors stem from attention to detail rather than fundamental reasoning failures. These patterns suggest that providing more structured guidance about database schemas and SQL syntax rules can effectively address a substantial portion of generation errors.

However, the moderate overall success rates of 43-49% also reveal significant limitations in current repair approaches. Complex reasoning errors, particularly those involving multi-table relationships or sophisticated aggregation requirements, prove resistant to correction through additional prompting alone. This finding indicates that future research should focus on developing more sophisticated error analysis and correction strategies that can address fundamental semantic misunderstandings rather than merely surface-level mistakes.

5.3 Implications and Limitations

5.3.1 Practical Deployment Considerations

The experimental results demonstrate performance levels that approach viability for practical deployment scenarios, though several considerations affect real-world applicability. The 80-84% exact match accuracy achieved by both models represents substantial capability for supporting database querying tasks, particularly in scenarios where users can validate and potentially modify generated queries before execution. The repair mechanism’s ability to recover 43-49% of initially failed queries provides meaningful enhancement to system reliability.

However, the question sensitivity findings introduce important caveats for user interface design and system deployment. The consistent performance degradation on improved questions suggests that systems should be designed to guide users toward effective question formulations rather than encouraging natural language elaboration that might degrade performance. This finding challenges conventional assumptions about human-AI interaction design and suggests that optimal performance may require users to adapt their communication patterns to model capabilities.

The computational overhead associated with repair processing creates trade-offs between accuracy improvement and response time. With repair demand ranging from 32-39% depending on configuration, systems must balance the benefits of improved accuracy against the costs of increased latency and computational resource utilization. These considerations become particularly important in high-volume deployment scenarios where rapid response times are critical for user experience.

5.3.2 Technical and Methodological Limitations

Several technical limitations constrain the current system’s performance and suggest areas requiring additional research attention. The repair success rate of approximately 43-49% indicates that roughly half of repair attempts fail to achieve successful correction, suggesting significant room for improvement in error analysis and correction strategies. Complex queries involving multiple table joins and nested operations remain challenging for automated repair mechanisms, indicating the need for more sophisticated approaches to semantic error correction.

The evaluation methodology, while comprehensive within its defined scope, encounters limitations that may affect generalizability to broader real-world scenarios. The strict exact match criteria may underestimate system performance by penalizing semantically correct but syntactically different queries. Future evaluations should explore semantic

equivalence checking and execution accuracy metrics that focus on result correctness rather than query structure matching.

The dependence on external API services for language model access creates operational considerations for production deployment, including cost management, availability guarantees, and data privacy concerns. The limited control over model updates and behavioral changes introduces uncertainty about long-term system stability and performance consistency.

5.3.3 Question Improvement Paradox Analysis

The counterintuitive finding that improved questions consistently performed worse than original formulations represents a significant challenge to conventional assumptions about optimal human-AI interaction. This paradox suggests that current language models may be overfitted to specific linguistic patterns encountered during training, making them sensitive to deviations from familiar question structures even when those deviations appear to improve clarity from a human perspective.

The pattern of decreasing performance with question improvement across both models indicates that this is not merely a limitation of smaller models but a systematic characteristic of current LLM architectures. The finding has important implications for training data curation, model development, and system design, suggesting that robustness to natural language variation remains a significant challenge for text-to-SQL systems.

Future research should investigate the specific linguistic features that contribute to performance degradation in improved questions, develop training approaches that enhance robustness to question reformulation, and explore system design strategies that can guide users toward effective query formulations while maintaining natural interaction patterns.

5.4 Future Research Directions

5.4.1 Technical Enhancement Opportunities

Several promising directions emerge from our findings for advancing text-to-SQL system capabilities beyond current performance levels. The development of more sophisticated error classification and correction strategies could improve repair effectiveness by providing more precise diagnosis of failure causes and targeted correction approaches. Implementation of semantic understanding enhancements specifically focused on complex query requirements could address current limitations in handling multi-table relationships and nested operations.

Investigation of ensemble approaches that combine predictions from multiple models or correction strategies could potentially improve both initial accuracy and repair success rates. Development of adaptive repair mechanisms that learn from successful correction patterns could enhance system performance over time. Additionally, exploration of domain-specific adaptation techniques could optimize system performance for particular database environments or application contexts.

5.4.2 Evaluation and Validation Extensions

Future research should expand evaluation methodologies to better capture system utility in real-world scenarios. Development of semantic equivalence evaluation metrics would provide more comprehensive assessment by recognizing functionally correct queries that differ syntactically from gold standards. Comprehensive user studies investigating practical usability, satisfaction, and learning outcomes would provide essential insights for optimizing human-AI interaction approaches.

Cross-linguistic evaluation capabilities would establish system applicability across diverse international contexts, while analysis of performance across various database domains could identify domain-specific optimization opportunities. Long-term system behavior studies would provide insights into adaptation and learning potential over extended deployment periods.

Chapter 6

Conclusion and Future Work

This chapter summarizes the key findings of our research on text-to-SQL systems with automated repair mechanisms, discusses our contributions to the field, and outlines future research directions based on our experimental results.

6.1 Research Summary

6.1.1 Problem Statement and Motivation

Current text-to-SQL systems face significant challenges in generating accurate SQL queries from natural language questions. Even advanced large language models frequently produce incorrect queries that require manual correction by users with technical expertise. Our research aimed to develop an automated repair system capable of detecting and correcting common SQL generation errors without human intervention, while systematically investigating how different question formulations affect system performance.

6.1.2 Our Six-Layer Solution Approach

We developed a comprehensive six-layer system architecture that incorporates an intelligent automated repair mechanism as Layer 5. The system operates through: (1) Input Layer for question preprocessing, (2) Prompt Engineering Layer for schema-aware prompt construction, (3) LLM Layer for initial SQL generation, (4) Execution Layer for database query execution, (5) Validation and Correction Layer for error detection and repair, and (6) Output Layer for result formatting.

The repair mechanism analyzes error types and user question intent through pattern recognition to guide SQL correction, classifies errors into schema violations, syntax errors,

and semantic logic errors, provides targeted hints to language models for correction guidance, and implements up to three iterative repair attempts per failed query.

6.2 Key Experimental Results

6.2.1 Comprehensive Model Performance Assessment

We evaluated our system on the Spider development dataset (1,034 questions) across three language models with significantly different capabilities. Table 6.1 summarizes our core findings.

Table 6.1: Final Performance Summary Across All Tested Models

Model	Original Questions	Improved Questions	Repair Success Rate
GPT-4o	866/1034 (83.8%)	846/1034 (81.8%)	49.4% / 47.5%
GPT-4o-mini	830/1034 (80.3%)	802/1034 (77.6%)	43.0% / 42.7%
LLaMA Coder	519/1034 (50.2%)	478/1034 (46.2%)	16.9% / 13.4%
Qwen Coder	465/1034 (45.0%)	445/1034 (43.0%)	4.8% / 3.4%

GPT-4o achieved the strongest baseline performance with 83.8% accuracy on original questions, demonstrating substantial capability in understanding database schemas and generating syntactically correct SQL queries. GPT-4o-mini achieved competitive 80.3% accuracy with a 3.5 percentage point gap reflecting the benefits of larger model parameters.

Qwen Coder demonstrated significantly lower performance at 45.0% accuracy, representing a 38.8 percentage point gap compared to GPT-4o. This substantial difference highlights the critical importance of model selection in text-to-SQL applications and demonstrates that not all language models are equally capable of structured data querying tasks.

6.2.2 Question Improvement Paradox Discovery

Our most significant and counterintuitive finding relates to the performance degradation observed when processing "improved" questions rather than original Spider formulations. All three models showed consistent performance decline: GPT-4o lost 20 correct answers (-1.9%), GPT-4o-mini lost 28 correct answers (-2.7%), and Qwen Coder lost 20 correct answers (-1.9%).

This finding directly contradicts expectations that clearer, more explicit question formulations should improve system performance. The consistent pattern across all models indicates systematic rather than random effects, suggesting that question reformulation may introduce linguistic patterns less familiar to model training data or create semantic

ambiguities that interfere with SQL mapping processes.

6.2.3 Repair Mechanism Effectiveness Analysis

The automated repair mechanism demonstrated effectiveness that strongly correlates with underlying model capabilities. GPT-4o achieved 49.4% repair success on original questions (164/332 successful repairs) and 47.5% on improved questions (170/358). GPT-4o-mini showed lower but meaningful effectiveness with 43.0% success on original questions (154/358) and 42.7% on improved questions (173/405).

Qwen Coder demonstrated severely limited repair effectiveness with only 4.8% success on original questions (29/598) and 3.4% on improved questions (21/610). The model required repair attempts on 57.8% of questions, indicating fundamental SQL generation deficiencies that the repair mechanism could not address.

These results prove that repair mechanisms require baseline model competence to function effectively and cannot serve as universal solutions for poor text-to-SQL performance.

6.3 Research Contributions

6.3.1 Automated Repair Mechanism for Text-to-SQL

We developed the first comprehensive intelligent repair mechanism specifically designed for text-to-SQL systems that demonstrates meaningful effectiveness across strong language models. The system automatically detects and corrects schema mismatches, syntax errors, and logical inconsistencies, achieving 43-49% repair success rates for capable models like GPT-4o and GPT-4o-mini.

The repair mechanism’s model-agnostic design ensures applicability across different language model architectures. However, our findings clearly demonstrate that repair effectiveness requires baseline model competence, as evidenced by the poor performance on Qwen Coder (4.8% success rate).

6.3.2 Question Sensitivity Analysis and Formulation Guidelines

We conducted the first systematic study investigating how question formulations impact text-to-SQL performance, revealing the counterintuitive finding that "improved" questions consistently degrade performance across all tested models. This discovery challenges prevailing assumptions about optimal human-AI interaction and reveals important limitations in current language models’ ability to generalize beyond training data patterns.

The finding has practical implications for user interface design, suggesting that systems should guide users toward question formulations that align with model training patterns rather than encouraging natural language elaboration that may impair performance.

6.3.3 Comprehensive Evaluation Framework

We established a thorough evaluation methodology that measures both baseline text-to-SQL performance and quantifies automated repair mechanism effectiveness across multiple models with different capability levels. This framework enables fair comparisons while providing detailed insights into error patterns, correction success rates, and the relationship between model capabilities and repair effectiveness.

6.4 Practical Implications and Deployment Considerations

6.4.1 Model Selection Guidelines

Our findings provide clear guidance for model selection in text-to-SQL applications. Organizations should prioritize high-capability models like GPT-4o over weaker alternatives, as performance differences are substantial (83.8% vs 45.0%) and repair mechanisms cannot bridge fundamental capability gaps. The moderate repair success rates for strong models (43-49%) justify implementation complexity, while poor repair performance on weak models (4.8%) confirms that repair systems cannot compensate for inadequate base capabilities.

6.4.2 System Architecture Recommendations

The six-layer architecture with integrated repair mechanisms provides a practical framework for text-to-SQL system implementation. The validation and correction layer should be implemented for systems using strong language models but may not justify complexity for systems using weaker models. The 32-39% repair demand rate creates computational overhead that must be considered in deployment scenarios balancing accuracy improvement against response time requirements.

6.5 Limitations and Constraints

6.5.1 Technical Limitations

The repair mechanism’s 43-49% success rate, while meaningful for strong models, indicates that approximately half of repair attempts still fail. Complex semantic errors, intricate multi-table reasoning problems, and highly ambiguous queries remain challenging for current repair approaches. The system’s performance remains fundamentally constrained by underlying language model capabilities, as repair mechanisms cannot overcome basic limitations in model understanding and reasoning.

The computational overhead introduced by repair processing affects 32-39% of queries, increasing both response time and API costs. This overhead becomes problematic for high-volume applications where rapid response is critical. Additionally, dependence on external API services creates operational risks related to availability, cost management, and limited control over model updates.

6.5.2 Evaluation and Generalization Limitations

Our evaluation used exclusively the Spider development dataset, which may not fully represent the complexity and diversity of queries in production environments. The strict exact match accuracy metric may underestimate practical utility by penalizing semantically correct but syntactically different SQL queries. The English-only focus limits direct applicability to multilingual environments, and transferability of question sensitivity findings to other languages remains unexplored.

6.6 Future Research Directions

6.6.1 Enhanced Repair Mechanism Development

Future research should focus on improving repair success rates beyond the current 43-49% ceiling through more sophisticated error classification mechanisms and advanced semantic understanding techniques. Investigating ensemble approaches that combine multiple correction strategies could provide greater robustness. Domain-specific adaptation methods such as few-shot learning could improve repair performance for particular database environments.

6.6.2 Question Formulation Optimization

The counterintuitive question formulation effects warrant dedicated investigation to understand underlying mechanisms and develop more robust systems. Future work should identify specific linguistic features that cause performance degradation and develop training approaches that improve model robustness to question variations. Comprehensive guidelines for optimal question design in text-to-SQL contexts could help users and system designers maximize performance.

6.6.3 Evaluation Framework Extensions

Future evaluations should expand beyond current limitations through testing on diverse datasets like Spider 2.0, BIRD, and KaggleDBQA to assess generalizability. Developing semantic equivalence metrics that recognize functionally correct answers regardless of syntactic differences would provide more realistic assessments. Conducting user studies to gauge practical usability and productivity impact would complement accuracy-focused evaluations.

6.6.4 Cross-Model Generalization Studies

Investigating repair mechanism effectiveness across a broader range of language models, including other commercial APIs and open-source alternatives, would provide deeper insights into the relationship between model capabilities and repair success. Understanding the minimum capability thresholds required for effective repair could guide model selection decisions in resource-constrained environments.

6.7 Final Assessment

Our research demonstrates both the substantial potential and persistent limitations of repair-enhanced text-to-SQL systems. The achievement of 80-83% accuracy for strong models showcases meaningful progress toward practical deployment in many scenarios. The repair mechanism’s consistent 43-49% success rate for capable models proves the value of intelligent error correction in improving system reliability.

However, the counterintuitive finding that improved questions paradoxically degrade performance reveals fundamental gaps in understanding human-AI interaction in specialized domains. This discovery challenges assumptions about making AI systems more user-friendly and highlights the intricate relationship between model training patterns

and optimal interaction strategies.

The path forward requires sustained effort in both technical advancement and thoughtful interaction design. While our system achieves performance levels approaching practical utility for strong models, significant challenges remain in maximizing error correction effectiveness, ensuring robustness to diverse question formulations, and addressing the fundamental capability gaps that prevent repair mechanisms from functioning effectively with weaker models.

Our work establishes a robust foundation for the next generation of text-to-SQL systems, providing comprehensive assessment of current capabilities and clear directions for future advancement in automated database querying technology. The demonstrated correlation between model capabilities and repair effectiveness provides important guidance for researchers and practitioners working to deploy these systems in real-world applications.

Chapter 7

Critical Reflection

This chapter provides a critical reflection on my personal learning journey throughout this research project, examining key insights gained, challenges encountered, and how this experience has shaped my understanding of text-to-SQL systems and research methodology.

7.1 Technical Learning and Development

This research significantly expanded my technical capabilities in several key areas. Working with large language model APIs taught me the practical challenges of integrating external services into research systems, including handling rate limits, managing costs, and ensuring reproducible results across different API versions. The experience of implementing iterative repair mechanisms provided deep insights into the complexities of error analysis and correction in natural language processing systems.

The most significant technical learning involved understanding the relationship between model capabilities and system effectiveness. Initially, I expected repair mechanisms to provide universal benefits across all models. However, the stark differences in repair success rates—from 49.4% for GPT-4o to 4.8% for Qwen Coder revealed that architectural innovations cannot compensate for fundamental model limitations. This taught me the importance of baseline competence thresholds in system design.

7.2 Research Methodology Insights

The research process highlighted the critical importance of systematic evaluation across multiple models rather than single-model assessments. My initial focus on GPT-4o alone would have missed the capability threshold phenomenon that emerged when testing weaker models. This experience reinforced the value of comprehensive experimental design in

drawing reliable conclusions.

The counterintuitive finding that "improved" questions decreased performance across all models taught me to question assumptions and remain open to unexpected results. This discovery challenged my preconceptions about question clarity and system performance, demonstrating that research often reveals outcomes contrary to intuitive expectations.

7.3 Challenges and Problem-Solving

The most significant challenge involved managing the complexity of multi-model evaluation while maintaining experimental rigor. Coordinating API access, ensuring consistent experimental conditions, and handling different model response formats required careful planning and systematic execution. Each model presented unique integration challenges that demanded adaptive problem-solving approaches.

Debugging repair mechanism failures proved particularly challenging, as it required understanding not just technical errors but also the semantic reasoning failures that repair systems could not address. This experience taught me the importance of comprehensive error analysis and the limitations of automated correction approaches.

7.4 Understanding of AI System Limitations

This research provided valuable insights into the current limitations of AI systems in practical applications. The 70-75% accuracy ceiling that persists across different text-to-SQL approaches, despite significant advances in language models, revealed the gap between research achievements and production requirements.

The question sensitivity findings particularly highlighted how current AI systems can be brittle in unexpected ways. The fact that semantically equivalent questions produced different results challenged my understanding of how these systems process natural language and emphasized the importance of robustness testing in AI system evaluation.

7.5 Future Research Perspective

This experience has shaped my approach to future research in several important ways. I now prioritize comprehensive multi-system evaluation over single-model optimization, recognizing that generalizability is crucial for meaningful research contributions. The importance of identifying system limitations alongside capabilities has become central to

my research philosophy.

The project also reinforced the value of honest reporting of negative results. The question improvement paradox and repair mechanism limitations provide valuable insights for the research community, even though they contradict initial expectations. This experience taught me that unexpected findings often contribute more to scientific understanding than confirmatory results.

7.6 Personal Growth and Skills Development

This research project significantly developed my skills in experimental design, data analysis, and scientific writing. Managing large-scale evaluations across multiple models required careful attention to detail and systematic record-keeping. The experience of presenting complex technical findings in accessible formats improved my ability to communicate research effectively.

Working with real-world datasets and production-scale systems provided valuable experience in handling the practical challenges of AI research beyond theoretical development. This included managing computational resources, handling API limitations, and ensuring reproducible results across different experimental conditions.

7.7 Reflection on Research Impact

The research contributes meaningfully to the text-to-SQL field by addressing a critical gap in error correction capabilities. However, the findings also reveal important limitations that guide realistic expectations for automated repair systems. The capability threshold discovery provides practical guidance for researchers and practitioners considering similar approaches.

The question sensitivity analysis opens new research directions that were previously unexplored, potentially influencing how future systems handle linguistic variation. This unexpected contribution demonstrates how systematic evaluation can reveal important phenomena that might otherwise remain hidden.

7.8 Lessons for Future Work

This research taught me several important lessons for future work. First, the importance of testing assumptions through systematic evaluation rather than relying on intuitive

expectations. Second, the value of comprehensive baseline comparisons to establish the true contribution of proposed innovations. Third, the necessity of honest reporting of both positive and negative results to advance scientific understanding.

The experience also emphasized the importance of considering practical deployment constraints alongside technical performance metrics. The gap between research achievements and production requirements highlighted the need for research that addresses real-world deployment challenges rather than optimizing purely for benchmark performance.

This research journey has fundamentally shaped my understanding of AI system development and evaluation, providing a solid foundation for future contributions to the field of natural language processing and database interfaces.

Bibliography

- [1] Yanbo Shen et al. “MapleRepair: A Study of In-Context-Learning-Based Text-to-SQL Errors”. In: *arXiv preprint* (2025). Forthcoming publication. URL: <https://arxiv.org/search/?query=MapleRepair&searchtype=all>.
- [2] Dingzirui Wang et al. “DAC: Decomposed Automation Correction for Text-to-SQL”. In: *arXiv preprint arXiv:2408.08779* (2024). URL: <https://arxiv.org/abs/2408.08779>.
- [3] Tao Yu et al. “Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task”. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2018, pp. 3911–3921. URL: <https://aclanthology.org/D18-1425/>.
- [4] Naihao Deng, Yuwei Chen, and Yue Zhang. “Structure-Grounded Pretraining for Text-to-SQL”. In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (2021), pp. 3335–3350. URL: <https://arxiv.org/pdf/2010.12773>.
- [5] Xiaowei Zhu et al. “Large Language Model Enhanced Text-to-SQL Generation: A Survey”. In: *arXiv preprint arXiv:2410.06011* (2024). URL: <https://arxiv.org/abs/2410.06011>.
- [6] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017, pp. 5998–6008. URL: <https://papers.nips.cc/paper/7181-attention-is-all-you-need>.
- [7] OpenAI. “GPT-4 Technical Report”. In: *arXiv preprint arXiv:2303.08774* (2023). URL: <https://arxiv.org/abs/2303.08774>.
- [8] Xiaowei Zhu et al. “Large Language Model Enhanced Text-to-SQL Generation: A Survey”. In: *arXiv preprint arXiv:2410.06011* (2024). URL: <https://arxiv.org/abs/2410.06011>.
- [9] Reza Mohammadjafari, Wei Chen, and Yang Liu. “From Natural Language to SQL: Review of LLM-based Text-to-SQL Systems”. In: *arXiv preprint arXiv:2410.01066* (2024). URL: <https://arxiv.org/html/2410.01066v1>.

- [10] Tom Brown et al. “Language Models are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 1877–1901. URL: <https://papers.nips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>.
- [11] Lei Wang et al. “TabSD: Large Free-Form Table Question Answering with SQL-Based Table Decomposition”. In: *arXiv preprint arXiv:2502.13422* (2025). URL: <https://arxiv.org/abs/2502.13422>.
- [12] Victor Zhong, Caiming Xiong, and Richard Socher. “Evaluating Text-to-SQL with Execution Semantics: Challenges and Solutions”. In: *arXiv preprint arXiv:2003.05444* (2020). URL: <https://arxiv.org/abs/2003.05444>.

Appendix A

System Configuration Parameters

This appendix provides the complete configuration parameters used in our text-to-SQL system evaluation.

Table A.1: Language Model API Configuration

Parameter	Value
Model (Primary)	GPT-4o
Model (Secondary)	GPT-4o Mini
Model (Tertiary)	LLaMA Coder
Temperature	0 (deterministic)
Max Tokens	512
Top-p	1.0
Frequency Penalty	0
API Timeout	30 seconds
Rate Limit Delay	0.2 seconds

Table A.2: Repair System Parameters

Parameter	Value
Maximum Repair Attempts	3
Question Analysis Enabled	True
Schema Validation	Enabled
Result Comparison	Exact Match
Error Classification	Multi-level
Repair History Tracking	Enabled

Table A.3: Database Execution Settings

Setting	Value
Database Engine	SQLite 3.x
Query Mode	Read-only (PRAGMA query_only = ON)
Connection Timeout	30 seconds
Result Display Limit	5 rows
Error Logging	Comprehensive
Connection Pooling	Disabled (isolated connections)

Appendix B

Detailed Experimental Results

B.1 Complete Performance Breakdown

Table B.1: Complete Multi-Model Performance Results

Model	Baseline	With Repair	Improvement	Repair Success
GPT-4o	63.0%	83.8%	+20.8 pts	49.4% (164/332)
GPT-4o Mini	57.0%	80.3%	+23.3 pts	43.0% (154/358)
LLaMA Coder	39.2%	49.3%	+10.1 pts	16.6% (104/628)

B.2 Question Type Performance Analysis

Table B.2: Original vs Improved Question Performance

Model	Original Questions	Improved Questions	Difference
GPT-4o	83.8%	81.8%	-1.9 pts
GPT-4o Mini	80.3%	77.6%	-2.7 pts
LLaMA Coder	49.3%	46.1%	-3.2 pts

B.3 Error Type Distribution

Table B.3: Error Categories and Repair Success Rates

Error Type	Frequency	Repair Success Rate	Difficulty
Schema Mismatches	35%	62%	Medium
Aggregation Errors	25%	58%	Medium
JOIN Problems	20%	41%	High
Syntax Errors	12%	75%	Low
Logic Errors	8%	23%	Very High

Appendix C

Question Intent Analysis Patterns

C.1 Keyword Pattern Recognition

Table C.1: Question Intent Keywords and Success Rates

Intent Category	Keywords	Repair Success Rate
COUNT Operations	"how many", "count", "total"	67%
AVERAGE Operations	"average", "mean", "avg"	56%
MAXIMUM Operations	"highest", "maximum", "largest"	61%
MINIMUM Operations	"lowest", "minimum", "smallest"	61%
GROUP BY Operations	"each", "every", "per", "by"	52%
ORDER BY Operations	"top", "bottom", "sort", "order"	45%
JOIN Operations	"and", "with", "both"	38%

C.2 Sample Repair Prompts

C.2.1 COUNT Operation Repair Example

Original Question: "How many students are enrolled?"

Failed SQL: `SELECT * FROM students WHERE enrolled = true;`

Error: Column 'enrolled' doesn't exist

Repair Hint: "This question asks for COUNT - use COUNT(*) or COUNT(column)"

Corrected SQL: `SELECT COUNT(*) FROM students;`

C.2.2 JOIN Operation Repair Example

Original Question: "Show students and their grades"

Failed SQL: `SELECT students.name, grades.grade FROM students, grades;`

Error: Cartesian product - missing JOIN condition

Repair Hint: "This question might require JOINS between tables"

Corrected SQL: `SELECT students.name, grades.grade`

`FROM students`

`JOIN grades ON students.student_id = grades.student_id;`

Appendix D

Spider Dataset Analysis

D.1 Dataset Statistics

Table D.1: Spider Dataset Detailed Statistics

Metric	Count
Total Development Questions	1,034
Development Databases	20
Training Databases	140
Zero Database Overlap	Yes
Average Question Length	13 words
Average SQL Length	21 tokens
Complex Queries (>2 tables)	312 (30.2%)
Simple Queries (1 table)	722 (69.8%)

D.2 Database Domain Distribution

Table D.2: Database Domains in Spider Dev Set

Domain Category	Number of Databases
Academic	5
Business	4
Entertainment	3
Sports	2
Government	2
Other	4
Total	20

Appendix E

Implementation Code Snippets

E.1 Core Repair Function

```
def predict_with_repairs(question: str, db_id: str, gold_result=None):
    """Enhanced prediction function with intelligent repair mechanism"""
    # Initial SQL generation
    sql = llm_generate_sql(question, db_id)
    out = run_sql(db_id, sql)
    attempts = 0
    repair_reasons = []
    sql_history = [sql]

    while attempts < MAX_REPAIRS:
        need_repair = False
        error_msg = None

        # Check for SQL execution errors
        if isinstance(out, str) and out.startswith("SQL ERROR"):
            need_repair = True
            error_msg = out
            repair_reasons.append(f"SQL Error")

        # Check for wrong results (if gold result is provided)
        elif gold_result is not None and not isinstance(out, str):
            exact, _, _, _ = compare(gold_result, out)
            if not exact:
                need_repair = True
```



```

        error_msg = generate_error_message(gold_result, out)
        repair_reasons.append(f"Wrong Result")

    if not need_repair:
        break

    # Attempt repair
    sql = llm_repair_sql(question, db_id, sql, error_msg, gold_result, out)
    sql_history.append(sql)
    out = run_sql(db_id, sql)
    attempts += 1
    time.sleep(SLEEP)

return sql, out, attempts, repair_reasons, sql_history

```

E.2 Question Intent Analysis Function

```

def analyze_question_intent(question: str, db_id: str) -> str:
    """Generate intelligent repair hints based on question analysis"""
    question_lower = question.lower()
    hints = []

    # Count-related patterns
    if any(word in question_lower for word in
           ['how many', 'count', 'number of', 'total']):
        hints.append("- This question asks for COUNT - use COUNT(*) or COUNT(column)")

    # Average patterns
    if any(word in question_lower for word in ['average', 'mean', 'avg']):
        hints.append("- This question asks for AVERAGE - use AVG(column)")

    # Maximum patterns
    if any(word in question_lower for word in
           ['highest', 'maximum', 'max', 'largest', 'most']):
        hints.append("- This question asks for MAXIMUM - use MAX(column)")

    return "\n".join(hints) if hints else "- Analyze the question structure carefully

```